

Lo primero que he de hacer es ejecutar.....	2
El segundo paso es to add the test project to your solution	5
En tercer lugar, add a reference from (and inside) the test project -> i.e. to the (backend) API project.....	7
To verify that everything is set up correctly, corremos dotnet build from the solution root (‘backend’ folder)	10
At this point, your test project has everything it needs to start unit testing the CRUD endpoints of ReservasController.	19
Entonces a estas alturas ya tenemos tests xunit exitosos para get api reservas y get api usuarios,	58

Tests unitarios de mi ASP.NET Core Web API backend con xUnit

En 1io, no necesito instalarme nada nuevo para hacer estos tests.

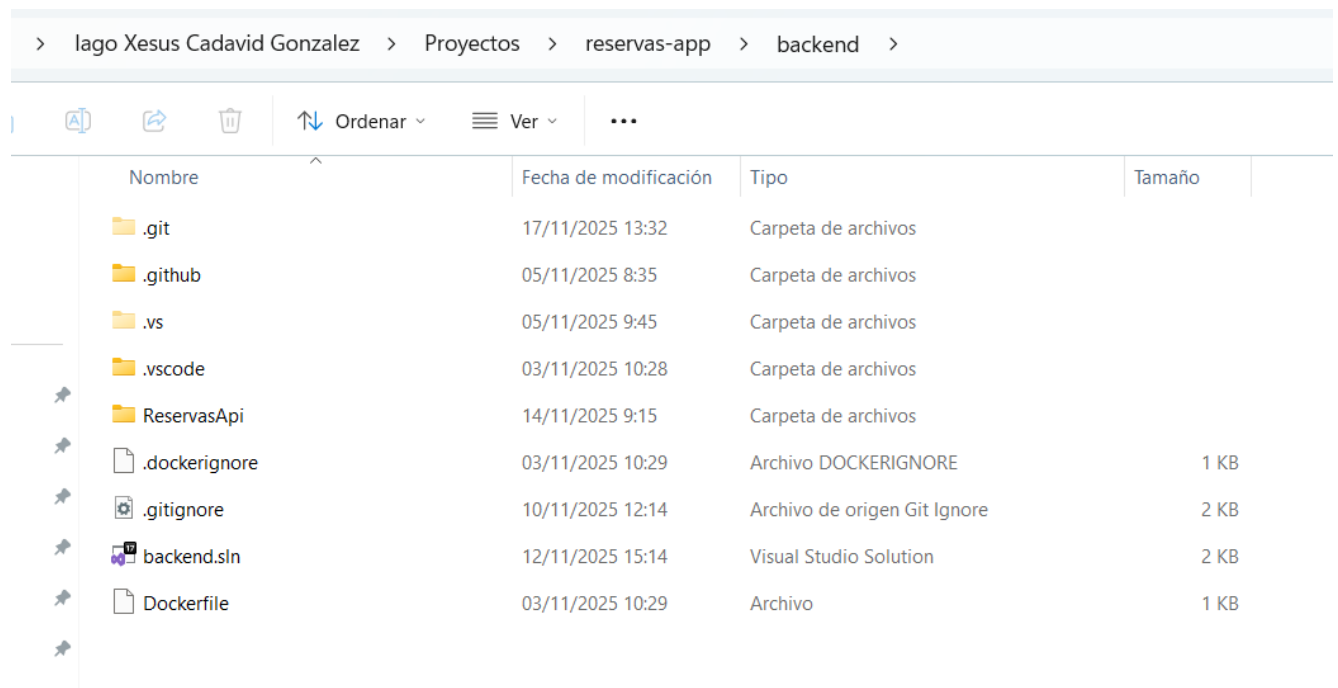
Lo primero que he de hacer es ejecutar:

dotnet new xunit -n MyBookingApi.Tests

That is “the **first step** to start adding unit tests to your project”.

Le pregunté “is the test project created based on my existing (backend) project or not?” y chatgpt me respondió: No, the xUnit test project is NOT created from your existing project. **It (the xUnit test project) is a separate, empty project that you connect to your main project afterward.**

“Running **the command** in the **wrong folder** creates the test project in the **wrong place**”. You should run it from the root folder of your (backend) solution, the same directory where your .sln file is located.



> Iago Xesus Cadavid Gonzalez > Proyectos > reservas-app > backend >					
	Nombre	Fecha de modificación	Tipo	Tamaño	
	.git	17/11/2025 13:32	Carpeta de archivos		
	.github	05/11/2025 8:35	Carpeta de archivos		
	.vs	05/11/2025 9:45	Carpeta de archivos		
	.vscode	03/11/2025 10:28	Carpeta de archivos		
	ReservasApi	14/11/2025 9:15	Carpeta de archivos		
	.dockerignore	03/11/2025 10:29	Archivo DOCKERIGNORE	1 KB	
	.gitignore	10/11/2025 12:14	Archivo de origen Git Ignore	2 KB	
	backend.sln	12/11/2025 15:14	Visual Studio Solution	2 KB	
	Dockerfile	03/11/2025 10:29	Archivo	1 KB	

“That (‘backend’ folder es la raíz de mi backend y es padre de backend.sln) means your *solution root* is the ‘backend’ folder, the one that contains backend.sln”

PÁGINA SIGUIENTE

Perfect — thanks for the clarification.

Your structure is now clearly:

```
markdown

backend/
  backend.sln
  ReservasApi/
    ReservasApi.csproj
```

This is the **standard and correct** structure for a .NET solution with a single project.

So let's walk through it again with this exact setup.

✓ Where to run the command

You should run:

```
bash

dotnet new xunit -n ReservasApi.Tests
```



from inside the `backend` folder, the one that contains `backend.sln`, like this:

If I open cmd from Windows file explorer, where would I open it from ?

Given your project structure:

```
markdown

backend/
  backend.sln
  ReservasApi/
    ReservasApi.csproj
```

 Copy code

If you open **Command Prompt (cmd)** from File Explorer, you should open it **from the** `backend` folder, the one that contains `backend.sln`.

✓ Why the `backend` folder

- This is your **solution root**.
- Running the commands here keeps the test project **next to your API project**, not inside it.
- It allows you to **add the test project to the solution** and reference your API project correctly.

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet new xunit -n ReservasApi.Tests
La plantilla "Proyecto de prueba de xUnit" se creó correctamente.

Procesando acciones posteriores a la creación...
Restaurando C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasApi.Tests.csproj:
Restauración realizada correctamente.
```

Ese comando **crea el xUnit test project**.

> Iago Xesus Cadavid Gonzalez > Proyectos > reservas-app > backend >				
Ordenar Ver				
Nombre		Fecha de modificación	Tipo	Tamaño
.git		17/11/2025 13:32	Carpeta de archivos	
.github		05/11/2025 8:35	Carpeta de archivos	
.vs		05/11/2025 9:45	Carpeta de archivos	
.vscode		03/11/2025 10:28	Carpeta de archivos	
ReservasApi		14/11/2025 9:15	Carpeta de archivos	
.dockerignore		03/11/2025 10:29	Archivo DOCKERIGNORE	1 KB
.gitignore		10/11/2025 12:14	Archivo de origen Git Ignore	2 KB
backend.sln		12/11/2025 15:14	Visual Studio Solution	2 KB
Dockerfile		03/11/2025 10:29	Archivo	1 KB
ReservasApi.Tests		21/11/2025 9:00	Carpeta de archivos	

Iago Xesus Cadavid Gonzalez > Proyectos > reservas-app > backend > ReservasApi.Tests >				
Ordenar Ver				
Nombre		Fecha de modificación	Tipo	Tamaño
obj		21/11/2025 9:00	Carpeta de archivos	
ReservasApi.Tests.csproj		21/11/2025 9:00	Archivo de origen C# Project	1 KB
UnitTest1.cs		21/11/2025 9:00	Archivo de origen C#	1 KB

un **ReservasControllerTests.cs** file.

A 27 de nov tmb tengo

PÁGINA SIGUIENTE

So the final structure becomes:

markdown

```
backend/  
  backend.sln  
  ReservasApi/  
    ReservasApi.csproj  
  ReservasApi.Tests/  
    ReservasApi.Tests.csproj
```

✓ This is exactly what you want.

El segundo paso es to [add the test project to your solution](#)

Still inside the backend folder i.e. habiendo abierto el cmd desde backend folder, ejecutamos:

dotnet sln add ReservasApi.Tests

Pero antes de hacerlo y por motivos de comparación después, vamos a ver que tengo ya ahora en el (backend) sln file, para ello lo abro en el bloc de notas:

```
Microsoft Visual Studio Solution File, Format Version 12.00  
# Visual Studio Version 17  
VisualStudioVersion = 17.5.2.0  
MinimumVisualStudioVersion = 10.0.40219.1  
Project("{FAE04EC0-301F-11D3-BF4B-00C04F79EFBC}") = "ReservasApi", "ReservasApi\ReservasApi.csproj", "{0428B5C6-9D5D-6F3A-0F4C-43576921DA8E}"  
EndProject  
Global  
  GlobalSection(SolutionConfigurationPlatforms) = preSolution  
    Debug|Any CPU = Debug|Any CPU  
    Release|Any CPU = Release|Any CPU  
  EndGlobalSection  
  GlobalSection(ProjectConfigurationPlatforms) = postSolution  
    {0428B5C6-9D5D-6F3A-0F4C-43576921DA8E}.Debug|Any CPU.ActiveCfg = Debug|Any CPU  
    {0428B5C6-9D5D-6F3A-0F4C-43576921DA8E}.Debug|Any CPU.Build.0 = Debug|Any CPU  
    {0428B5C6-9D5D-6F3A-0F4C-43576921DA8E}.Release|Any CPU.ActiveCfg = Release|Any CPU  
    {0428B5C6-9D5D-6F3A-0F4C-43576921DA8E}.Release|Any CPU.Build.0 = Release|Any CPU  
  EndGlobalSection  
  GlobalSection(SolutionProperties) = preSolution  
    HideSolutionNode = FALSE  
  EndGlobalSection  
  GlobalSection(ExtensibilityGlobals) = postSolution  
    SolutionGuid = {E3B00B15-7A4A-435D-B5BE-3B45232BF96E}  
  EndGlobalSection  
EndGlobal
```

Centrarme en la línea **Project("{GUID}") = etc., "{PROJECT-GUID}"**

A ese respecto, **“each Project(...) block represents a project included in your solution”**, since there is only one *block* for ReservasApi, that’s the only project currently in the solution.

Podemos ver lo mismo using the dotnet CLI to list projects:

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet sln list
Proyectos
-----
ReservasApi\ReservasApi.csproj
```

Pues bien, vamos ahora con:

habiendo abierto el cmd desde backend folder, ejecutamos:

dotnet sln add ReservasApi.Tests

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet sln list
Proyectos
-----
ReservasApi\ReservasApi.csproj

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet sln add ReservasApi.Tests
Se ha agregado el proyecto "ReservasApi.Tests\ReservasApi.Tests.csproj" a la solución.

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet sln list
Proyectos
-----
ReservasApi.Tests\ReservasApi.Tests.csproj
ReservasApi\ReservasApi.csproj

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>
```

En tercer lugar, **add a reference from (and inside) the test project -> i.e. to the (backend) API project**

Since the API project is in ReservasApi/ReservasApi.csproj, run:

dotnet add ReservasApi.Tests reference ReservasApi (once again we run it from the solution root i.e. the 'backend' folder that contains backend.sln).

This **links the test project to the (backend) API code.**

Pero nuevamente vamos a ver primero what project references are already added to my (backend) API project (ReservasApi.csproj) before adding the test project reference (to the API project). * (2 págs. después)

Para ello, en un editor de texto abrimos el .csproj file (hijo de ReservasApi) and look for **<ItemGroup>** sections that contain **<ProjectReference>** tags.

Mi **ReservasApi.csproj** pre-añadir la referencia del test project: NEXT PAGE

ReservasApi.csproj X

C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi > ReservasApi.csproj

```
1 <Project Sdk="Microsoft.NET.Sdk.Web">
2
3   <PropertyGroup>
4     <TargetFramework>net9.0</TargetFramework>
5     <Nullable>enable</Nullable>
6     <ImplicitUsings>enable</ImplicitUsings>
7   </PropertyGroup>
8
9   <ItemGroup>
10    <PackageReference Include="Confluent.Kafka" Version="2.12.0" />
11    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="9.0.10" />
12    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="9.0.10" />
13    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="9.0.10">
14      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
15      <PrivateAssets>all</PrivateAssets>
16    </PackageReference>
17    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="9.0.10" />
18    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="9.0.10">
19      <PrivateAssets>all</PrivateAssets>
20      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
21    </PackageReference>
22    <PackageReference Include="Swashbuckle.AspNetCore" Version="9.0.6" />
23  </ItemGroup>
24
25 </Project>
26
```

In your file:

xml

Copy code

```
<ItemGroup>
  <PackageReference Include="Confluent.Kafka" Version="2.12.0" />
  <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="9.0.10" />
  <PackageReference Include="Microsoft.EntityFrameworkCore" Version="9.0.10" />
  <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="9.0.10">...</PackageReference>
  <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="9.0.10" />
  <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="9.0.10">...</PackageReference>
  <PackageReference Include="Swashbuckle.AspNetCore" Version="9.0.6" />
</ItemGroup>
```

✓ Count of Project References: 0

✓ Count of NuGet Package References: 7

So your API project currently does not reference any other project, only external NuGet packages.

This means it's safe to add your test project reference (`dotnet add ReservasApi.Tests reference ReservasApi`) without worrying about conflicts.

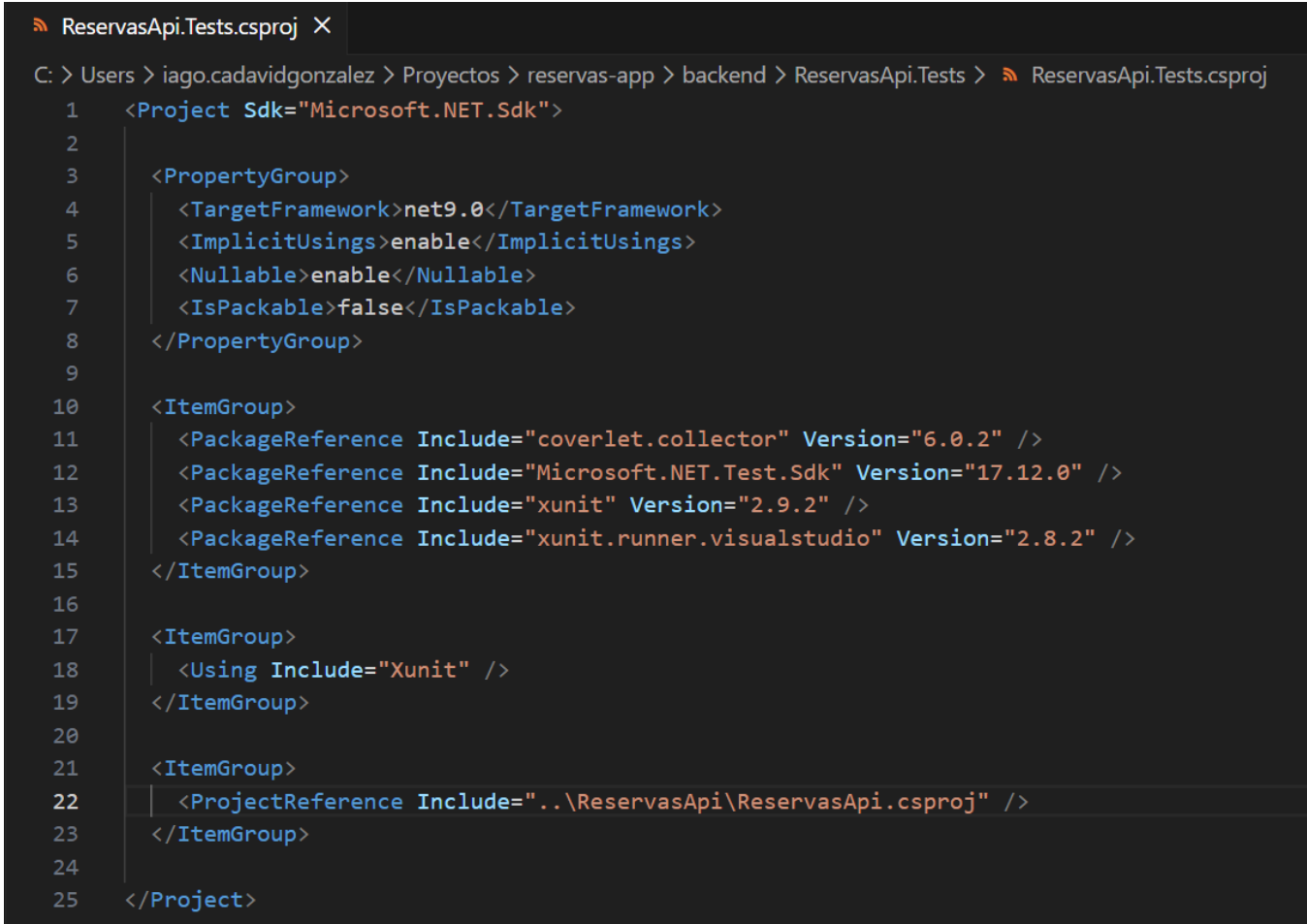
Nótese que ahí en **negrita** pone al API

project como sujeto activo de referenciar VS antes más bien dijimos que es lo referenciado.

Bueno, vamos a añadir ahora la referencia y ver que aparece en ReservasApi.csproj :

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet add ReservasApi.Tests reference ReservasApi
Se ha agregado la referencia "..\ReservasApi\ReservasApi.csproj" al proyecto.
```

Resulta que si ahora voy a **ReservasApi.csproj**, sigo viendo lo mismo que antes o sea NO reference, *pero esto se debe a que **donde realmente aparece esta refe que acabamos de añadir es en ReservasApi.Tests.csproj**:



```
ReservasApi.Tests.csproj X
C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi.Tests > ReservasApi.Tests.csproj
1  <Project Sdk="Microsoft.NET.Sdk">
2
3      <PropertyGroup>
4          <TargetFramework>net9.0</TargetFramework>
5          <ImplicitUsings>enable</ImplicitUsings>
6          <Nullable>enable</Nullable>
7          <IsPackable>false</IsPackable>
8      </PropertyGroup>
9
10     <ItemGroup>
11         <PackageReference Include="coverlet.collector" Version="6.0.2" />
12         <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
13         <PackageReference Include="xunit" Version="2.9.2" />
14         <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
15     </ItemGroup>
16
17     <ItemGroup>
18         <Using Include="Xunit" />
19     </ItemGroup>
20
21     <ItemGroup>
22         <ProjectReference Include="..\ReservasApi\ReservasApi.csproj" />
23     </ItemGroup>
24
25 </Project>
```

Porque el comando **dotnet add ReservasApi.Tests reference ReservasApi** means:

“Add a reference to the ReservasApi project inside the ReservasApi.Tests project” (inside se refiere al add)

Lo siguiente parece ser que es **“generate the first test file for your ReservasApi.project”** y **“test your first controller or service”**.

Since **I’ve linked my test project (ReservasApi.Tests) to my API (ReservasApi)**, now my **test project can see all my controllers, services and models** (los de ReservasApi backend project).

To verify that everything is set up correctly, corremos **dotnet build** from the solution root ('backend' folder)

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet add ReservasApi.Tests reference ReservasApi
Se ha agregado la referencia "..\ReservasApi\ReservasApi.csproj" al proyecto.

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet build
Restauración completada (6,7s)
  ReservasApi realizado correctamente (38,4s) → ReservasApi\bin\Debug\net9.0\ReservasApi.dll
  ReservasApi.Tests realizado correctamente (7,2s) → ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
Compilación realizado correctamente en 52,4s

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet sln list
Proyectos
-----
ReservasApi.Tests\ReservasApi.Tests.csproj
ReservasApi\ReservasApi.csproj

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>
```

If there are no errors, your **test project** (ReservasApi.Tests) correctly **references** your **API** (ReservasApi).

Now let's carefully go through how to **determine whether your controllers depend on interfaces or repositories**, because **this affects how you write unit tests** and **whether you need mocks**.

```
public ReservasController(AppDbContext contexto, IServicioProductorKafka productorKaf
```

[Copy code](#)

1 AppDbContext contexto

- Type: `AppDbContext` (concrete class)
- Meaning: Your controller is directly depending on the EF Core database context.
- Implication for unit testing:
 - You cannot easily mock `AppDbContext` directly.
 - If you want to unit test controller logic without touching a real database, you should **refactor to use a repository or service interface** that wraps database calls, e.g., `IBookingRepository`.
 - Alternative: you can use an **in-memory database** (`UseInMemoryDatabase`) for lightweight integration-style tests, but that's technically not a pure unit test.

Hablando con Victor y Sergio concluí usar `UseInMemoryDatabase` y así fue como testeé `get api reservas`, cuyo resultado es actual: 2 VS expected: 0 y por ende falla el test en su última línea de código. Dice Victor de debuggear poniendo puntos de ruptura para identificar qué falla exactamente. Recomiende saberse las teclas `f` tal `f` cual que `vs ide` usa para esto.

NEXT PAGE

2 IServicioProductorKafka productorKafka

- **Type:** `IServicioProductorKafka` (interface)
- **Meaning:** Your controller depends on an interface.
- **Implication for unit testing:**
 -  You can mock this dependency easily using Moq.
 - Example:

csharp

```
var mockKafka = new Mock<IServicioProductorKafka>();  
mockKafka.Setup(k => k.EnviaMensaje(It.IsAny<string>())).Verifiable();
```

F

The rest of my controllers do not have `IServicioProductorKafka` `productorKafka` as constructor parameter, they only have `AppDbContext` context. And to be honest, I don't care about Kafka right now, I mean my Kafka implementation is not completely working correctly as of now and I don't care about it, what I really want is (if possible) to unit test my CRUD endpoints (which are defined in the controllers) using xUnit.

Perfect — now we're very clear.

Your goal is: **unit test your CRUD endpoints in controllers that only depend on** `AppDbContext`, ignoring Kafka.

This is very doable, but we need to consider one key point:

1 Why controllers depending directly on `AppDbContext` is tricky for "pure" unit tests

- Unit tests should **not hit a real database**.
- `AppDbContext` is a concrete EF Core class, so your controller's methods will try to talk to a database if you instantiate it normally.
- To avoid hitting a real SQL Server, you have two options:

Antes de ir con las 2 opciones, muestro la aclaración que le hice sobre "your goal is:..."

"Your goal is: unit test your CRUD endpoints in controllers that only depend on ApplicationDbContext, ignoring Kafka." I also wanna test the CRUD endpoints of ReservasController, but ignoring (if possible) its 'productorKafka' constructor parameter.

Perfect — we can absolutely do that. You can **unit test** `ReservasController` **CRUD endpoints** while ignoring `IServicioProductorKafka productorKafka`.

Here's how:

1 Use Moq to mock Kafka

Since `productorKafka` is an interface, you can provide a **mock object** that does nothing. This way, your controller can be instantiated, but no real Kafka calls happen.

```
csharp
using Moq;
using ReservasApi.Controllers;

var mockKafka = new Mock<IServicioProductorKafka>();
```

 Copy code

Then pass it to the controller constructor:

```
csharp
var controller = new ReservasController(context, mockKafka.Object);
```

 Copy code

- `context` is your in-memory `AppDbContext` (for unit testing database logic).
- `mockKafka.Object` satisfies the constructor parameter without sending messages anywhere.

✓ This allows you to focus purely on **CRUD logic**.



Finalmente decidí simplemente comentar el parametro de la interfaz kafka que tenía como segundo param del constructor, quedando solo el param appdbcontext.

Ahora sí, vamos con las 2 opcns que chatgpt me da “to **avoid hitting a real sql server**”:

NEXT PAGE OPTION 1

Option A — Use EF Core In-Memory Database

- EF Core provides `UseInMemoryDatabase` which simulates a database in memory.
- Your controller code works as-is, but the data exists only in memory for the duration of the test.
- This is **the most common approach for CRUD unit tests** when you don't want Kafka or a real DB.

Example setup for `AppDbContext` :

```
csharp Copy  
  
var options = new DbContextOptionsBuilder<AppDbContext>()  
    .UseInMemoryDatabase(databaseName: "TestDb")  
    .Options;  
  
using var context = new AppDbContext(options);
```

- You can seed data into `context` for your tests.
- Call your controller methods with this `context`.
- Assert results.

This way, you can test CRUD logic **without touching**  `Server` or `Docker`.

“without touching SQL Server

or Docker.”

Pero recordar que antes dijo que esa opción NO es test unitarios PUROS.

IMPORTANTE: **ese código va en `ReservasControllerTests.cs`, hijo de `ReservasApi.Tests` project.**

Option B: **Refactor to a repository or service interface** (“you should refactor to use a repo or service interface that wraps db calls”, dijimos antes):

- Create an interface like `IReservasRepository` that wraps CRUD operations.
- Controller (constructor) depends on the interface instead of `AppDbContext`.
- In unit tests, you mock the interface using `Moq`.

Advantage of option B is: true unit test with no database involved.

Disadvantage: requires refactoring existing controllers.

Vamos a hacer Option A que le vale a Victor y es más sencilla que la option B.

Pero antes de ponerme con Option A lo que hago es comentar todas las líneas de código Kafka o sea la de `Program.cs`, la de `ReservasController`, y todas las de `ProductorKafka.cs` y la interfaz que implementa (a 27 de nov de 2025 tmb quité kafka del `docker-compose.yml`, pasándolo a un txt por si quiero volver a ponerlo). Pa que no me interfiera el Kafka con los test unitarios CRUD que voy a hacer con `xUnit` y `UseInMemoryDatabase`. Sergio dijo que hizo lo mismo, comentó Kafka y empleó `UseInMemoryDatabase`.

Since Kafka is completely removed, your ReservasController now only depends on ApplicationDbContext (ya que es el único param de su constructor). That means you are ready to **write xUnit tests using EFC In-Memory Database**.

First of all, make sure your test project is ready i.e. you should already have ReservasApi.Tests created and added to the solution (indeed lo tengo), also ensure it references your API project (indeed ya ejecuté “dotnet add ReservasApi.Tests reference ReservasApi”),

Now make sure you have xUnit installed i.e. package **xUnit** and package **xunit.runner.visualstudio** :

```
ReservasApi.Tests.csproj X
C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi.Tests > ReservasApi.Tests.csproj
1  <Project Sdk="Microsoft.NET.Sdk">
2
3    <PropertyGroup>
4      <TargetFramework>net9.0</TargetFramework>
5      <ImplicitUsings>enable</ImplicitUsings>
6      <Nullable>enable</Nullable>
7      <IsPackable>false</IsPackable>
8    </PropertyGroup>
9
10   <ItemGroup>
11     <PackageReference Include="coverlet.collector" Version="6.0.2" />
12     <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
13     <PackageReference Include="xunit" Version="2.9.2" />
14     <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
15   </ItemGroup>
16
17   <ItemGroup>
18     <Using Include="Xunit" />
19   </ItemGroup>
20
21   <ItemGroup>
22     <ProjectReference Include="..\ReservasApi\ReservasApi.csproj" />
23   </ItemGroup>
24
25 </Project>
26
```

When you run **dotnet new xunit -n ReservasApi.Tests** (que ya ejecuté anteriormente), the **xUnit project template automatically adds the xunit and xunit.runner.visualstudio packages** to the new project (i.e. the test project ReservasApi.Tests).

You do need to manually install **Microsoft.EntityFrameworkCore.InMemory** because that is NOT included in the xUnit template –you will use it for your **in-memory EFCore tests**. Para ello ejecutamos from the root folder

of my solution i.e. the directory where my backend.sln file is located, dicho directory en mi caso es la folder 'backend':

dotnet add ReservasApi.Tests package Microsoft.EntityFrameworkCore.InMemory

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet add ReservasApi.Tests package Microsoft.EntityFrameworkCore.InMemory

Compilación realizado correctamente en 0,6s
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "Microsoft.EntityFrameworkCore.InMemory" al proyecto "C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasApi.Tests.csproj".
info : GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/index.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/index.json 150 ms
info : GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/0.0.1-alpha/3.1.2.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/0.0.1-alpha/3.1.2.json 140 ms
info : GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/3.1.3/6.0.0-preview.6.21352.1.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/3.1.3/6.0.0-preview.6.21352.1.json 140 ms
info : GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/6.0.0-preview.7.21378.4/7.0.17.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/6.0.0-preview.7.21378.4/7.0.17.json 147 ms
info : GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/7.0.18/10.0.0-rc.2.25502.107.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/7.0.18/10.0.0-rc.2.25502.107.json 139 ms
info : GET https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/10.0.0/10.0.0.json
info : OK https://api.nuget.org/v3/registration5-gz-semver2/microsoft.entityframeworkcore.inmemory/page/10.0.0/10.0.0.json 139 ms
info : Restaurando paquetes para C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasApi.Tests.csproj...
info : GET https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.inmemory/index.json
info : OK https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.inmemory/index.json 164 ms
info : GET https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.inmemory/10.0.0/microsoft.entityframeworkcore.inmemory.10.0.0.nupkg
info : OK https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.inmemory/10.0.0/microsoft.entityframeworkcore.inmemory.10.0.0.nupkg 49 ms
info : Se ha instalado Microsoft.EntityFrameworkCore.InMemory 10.0.0 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\packages\microsoft.entityframeworkcore.inmemory\10.0.0 con el hash de contenido 06yN\NR9bOM7GjUR1hnucciik8nrr0bs7MP6RKCDHQP5BCcR4P9zemtlz6aznJms1xvybPzKwESTtoC6XMAPQ
Q==.
info : GET https://api.nuget.org/v3/vulnerabilities/index.json
info : OK https://api.nuget.org/v3/vulnerabilities/index.json 153 ms
info : GET https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json
info : GET https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.21.05.31.46/vulnerability.update.json
info : OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json 135 ms
info : OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.21.05.31.46/vulnerability.update.json 176 ms
error: NU1202: El paquete Microsoft.EntityFrameworkCore.InMemory 10.0.0 no es compatible con net9.0 (.NETCoreApp,Version=v9.0). El paquete Microsoft.EntityFrameworkCore.InMemory 10.0.0 admite: net10.0 (.NETCoreApp,Version=v10.0)
error: El paquete "Microsoft.EntityFrameworkCore.InMemory" no es compatible con los marcos de trabajo de "all" del proyecto "C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasApi.Tests.csproj".
```

Ese error es porque “you are using .NET 9 (net9.0) for your projects, **but you tried to install EF Core In-Memory 10.0.0, which only supports .NET 10**, that is why NuGet reports “error: NU1202:...**Microsoft.EFC.InMemory 10.0.0 NO es compatible con net 9.0”**”.

To fix it, you need to **install a version of M.EFC.InMemory that supports .NET 9**, so run:

dotnet add ReservasApi.Tests package Microsoft.EntityFrameworkCore.InMemory **--version 9.0.10**

--version 9.0.10 is EF Core 9 stable version **compatible with .NET 9 (net9.0)**. This will work with your current projects without upgrading to .NET 10.

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet add ReservasApi.Tests package Microsoft.EntityFrameworkCore.InMemory --version 9.0.10

Compilación realizado correctamente en 0,6s
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "Microsoft.EntityFrameworkCore.InMemory" al proyecto "C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasApi.Tests.csproj".
info : Restaurando paquetes para C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasApi.Tests.csproj...
info : GET https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.inmemory/index.json
info : OK https://api.nuget.org/v3-flatcontainer/microsoft.entityframeworkcore.inmemory/9.0.10/microsoft.entityframeworkcore.inmemory.9.0.10.nupkg 213 ms
info : Se ha instalado Microsoft.EntityFrameworkCore.InMemory 9.0.10 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\packages\microsoft.entityframeworkcore.inmemory\9.0.10 con el hash de contenido f8nQ7qcWlB3aFrantw9KcDjvYvwb+FLB3ZDM\EDC7uT9TJ3FPB1SaFbhuw2J1FPBw3fgr9IGayvaR2DQ9==.
info : GET https://api.nuget.org/v3/vulnerabilities/index.json
info : OK https://api.nuget.org/v3/vulnerabilities/index.json 153 ms
info : GET https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json
info : GET https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.21.05.31.46/vulnerability.update.json
info : OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json 135 ms
info : OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.21.05.31.46/vulnerability.update.json 176 ms
info : El paquete "Microsoft.EntityFrameworkCore.InMemory" es compatible con todos los marcos de trabajo especificados del proyecto "C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasApi.Tests.csproj".
info : Se agregó PackageReference para la versión "9.0.10" del paquete "Microsoft.EntityFrameworkCore.InMemory" al archivo "C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasApi.Tests.csproj".
info : Escribiendo el archivo de recursos en el disco. Ruta de acceso: C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\obj\project.assets.json
log : Se ha restaurado C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasApi.Tests.csproj (en 779 ms)
```

“El paquete **es compatible con todos** los marcos de trabajo del proyecto.

Se agregó PackageRefrence para la versión 9.0.10 del paquete al archivo ReservasApi.Tests.csproj”.

NEXT PAGE

```
ReservasApi.Tests.csproj X
C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi.Tests > ReservasApi.Tests.csproj
1 <Project Sdk="Microsoft.NET.Sdk">
2
3   <PropertyGroup>
4     <TargetFramework>net9.0</TargetFramework>
5     <ImplicitUsings>enable</ImplicitUsings>
6     <Nullable>enable</Nullable>
7     <IsPackable>false</IsPackable>
8   </PropertyGroup>
9
10  <ItemGroup>
11    <PackageReference Include="coverlet.collector" Version="6.0.2" />
12    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="9.0.10" />
13    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.12.0" />
14    <PackageReference Include="xunit" Version="2.9.2" />
15    <PackageReference Include="xunit.runner.visualstudio" Version="2.8.2" />
16  </ItemGroup>
17
18  <ItemGroup>
19    <Using Include="Xunit" />
20  </ItemGroup>
21
22  <ItemGroup>
23    <ProjectReference Include="..\ReservasApi\ReservasApi.csproj" />
24  </ItemGroup>
25
26 </Project>
```

At this point, your test project has everything it needs to start unit testing the CRUD endpoints of ReservasController.

Now it's time to **create a new C# file inside ReservasApi.Tests** i.e. create a test class, call it **ReservasControllerTests.cs**, this class will contain all your unit tests for CRUD endpoints (of ReservasController).

```
ReservasControllerTests.cs X ReservasController.cs
ReservasApi.Tests
1 using Microsoft.AspNetCore.Mvc;
2 using Microsoft.EntityFrameworkCore;
3 using ReservasApi.Controllers;
4 using ReservasApi.Data;
5 using ReservasApi.Modelos;
6 using System;
7 using System.Collections.Generic;
8 using System.Threading.Tasks;
9 using Xunit;
10
11 // using System.Linq;
12 // using System.Text;
13
```

Y using ReservasApi.Modelos.DTO. Sigue en página siguiente


```

namespace ReservasApi.Tests
{
    0 references
    public class ReservasControllerTests
    {
        // Helper method to create a new in-memory DbContext
        // Set up In-Memory AppDbContext for tests
        // Each test should use a fresh in-memory database to avoid state sharing between tests.
        1 reference
        private AppDbContext GetInMemoryDbContext(string dbName) {
            var opciones = new DbContextOptionsBuilder<AppDbContext>()
                .UseInMemoryDatabase(databaseName: dbName) // dbName can be unique per test ("TestDb1", "TestDb2")
                .Options;
            return new AppDbContext(opciones);
        } // todo este metodo privado aparece greyed out while not yet used anywhere

        // Test GET all reservas
        [Fact] // each test method must have the [Fact] attribute for xUnit to recognize it.
        0 references
        public async Task GetReservas_ReturnaAllReservas() // the test method must be inside the class that is marked with [Fact]
        {
            // Arrange:
            using var contexto = GetInMemoryDbContext("TestDb1"); // este método retorna un 'AppDbContext', que es un DbContext
            contexto.Reservas.Add(new Reserva { ReservaId = 1, Dni = "12349876F", InstalacionId = 1, NumeroAsistentes = 2 });
            contexto.Reservas.Add(new Reserva { ReservaId = 2, Dni = "67895432C", InstalacionId = 2, NumeroAsistentes = 3 });
            contexto.SaveChanges();

            // instancio ReservasController pasándole el 'contexto' que justo antes inicialicé con el retorno de GetInMemoryDbContext
            var controlador = new ReservasController(contexto); // 'contexto' es de tipo AppDbContext, que es el tipo que necesita ReservasController

            // Act:
            var resultado = await controlador.GetReservas(); // GetReservas lo tengo definido en ReservasController

            // Assert:
            var okResultado = Assert.IsType<OkObjectResult>(resultado.Result);
            var reservas = Assert.IsAssignableFrom<List<Reserva>>(okResultado.Value);
            Assert.Equal(2, reservas.Count);
        } // final de [Fact] GetReservas_ReturnaAllReservas()

        // You can add more [Fact] methods here for POST, GET by ID, PUT, DELETE
    }
}

```

Nº asistentes puse primero 2 y luego 3, nótese que **no puse FechaHora**, dijo chatgpt que no hacía falta, a pesar de estar en mi modelo de reservas.

You now have a test class, a working in-memory DbContext and your first test method *scaffolded*.

Pero dice que cuando me dio ese código “you had not yet shown me your actual GetReservas() (Reservas)controller code, so I gave you the *generic* pattern used when a controller returns entities directly”.

Pero como ahora sí conoce mi código concreto de ese GetReservas(), dice que debo reemplazar la parte del Assert osease cambiarle un poco el código, quedando la cosa así:

NEXT PAGE

✓ Step 1 — Fix your GET all reservas test

Replace this part:

```
csharp

var okResult = Assert.IsType<OkObjectResult>(result.Result);
var reservas = Assert.IsAssignableFrom<List<Reserva>>(okResult.Value);
Assert.Equal(2, reservas.Count);
```

with this:

```
csharp

var okResult = Assert.IsType<OkObjectResult>(result.Result);

var reservas = Assert.IsAssignableFrom<List<ReservaLeerDto>>(okResult.Value);
Assert.Equal(2, reservas.Count);
```

Because your controller maps each `Reserva` → `ReservaLeerDto` before returning.

⚠ One more important fix

Your current test creates **only a Reserva**, but your controller requires:

- A `Usuario` with matching `Dni`
- An `Instalacion` with matching `InstalacionId`

Example:

```
csharp

var reservas = await _contexto.Reservas
    .Include(r => r.Usuario)
    .Include(r => r.Instalacion)
    .ToListAsync();
```

Copy code

If you don't include related entities, your test will still pass, but properties in the DTO may be `null`.

That's acceptable for unit testing GET behavior — but I can help seed full data later if you want stricter tests.

Ese código del “Example” está en mi get api reservas de `ReservasController.cs`.

De momento no hago ese fix ya que dice “your test will still pass”.

! Step 2 — Try running the test

From your **backend folder**, run:

```
bash  
  
dotnet test
```

Expected output:

```
nginx  
  
Passed ReservasControllerTests.GetReservas_ReturnsAllReservas
```

If the test passes → we proceed to the next endpoint

If it fails, tell me the error message and I'll fix the test. 🙏

✓ Where should you run `dotnet test`?

You should run:

```
bash  
  
dotnet test
```

from the **solution root folder** — the folder that contains:

```
ReservasApi.sln  
ReservasApi/  
ReservasApi.Tests/
```



This is usually the *parent* folder of both your API project and your test project.

O sea que en mi caso corro dotnet test desde 'backend' folder:

NEXT PAGE

> Iago Xesus Cadavid Gonzalez > Proyectos > reservas-app > backend >

	Nombre	Fecha de modificación	Tipo	Tamaño
	.git	17/11/2025 13:32	Carpeta de archivos	
	.github	05/11/2025 8:35	Carpeta de archivos	
	.vs	05/11/2025 9:45	Carpeta de archivos	
	.vscode	03/11/2025 10:28	Carpeta de archivos	
📌	ReservasApi	21/11/2025 11:38	Carpeta de archivos	
📌	ReservasApi.Tests	21/11/2025 13:34	Carpeta de archivos	
📌	.dockerignore	03/11/2025 10:29	Archivo DOCKERIGNORE	1 KB
📌	.gitignore	10/11/2025 12:14	Archivo de origen Git Ignore	2 KB
📌	backend.sln	21/11/2025 9:20	Visual Studio Solution	3 KB
📌	Dockerfile	03/11/2025 10:29	Archivo	1 KB

Before running dotnet test, shouldn't I first do some sort of build for the changes (the testing code we've written) to take effect ?

Great question — and the answer is: NO, you do not need to build first.

Here's why:

✅ dotnet test automatically builds your solution

When you run:

```
bash
dotnet test
```

[Copy code](#)

.NET automatically performs these steps in order:

1. Restores packages (if needed)
2. Builds the solution/projects
3. Discovers the tests
4. Runs the tests

So you do **not** need to run:

```
nginx
dotnet build
```

[Copy code](#)

before dotnet test.



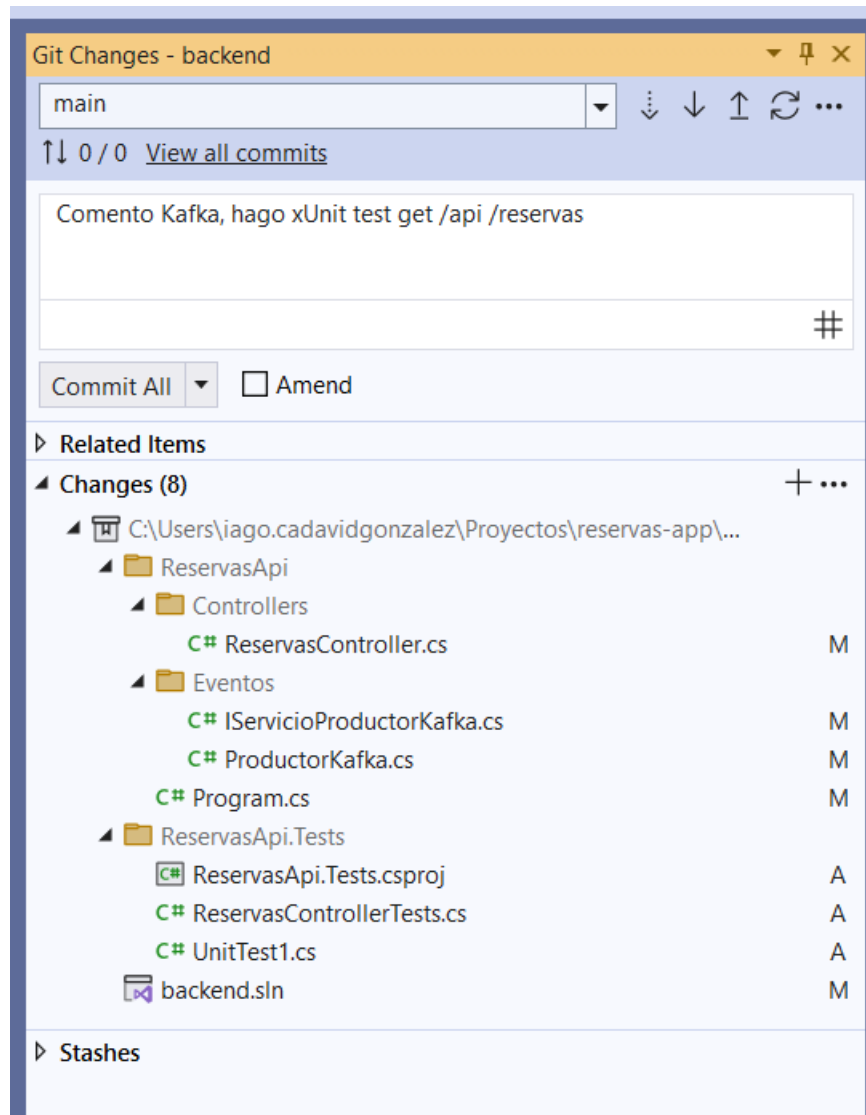
```

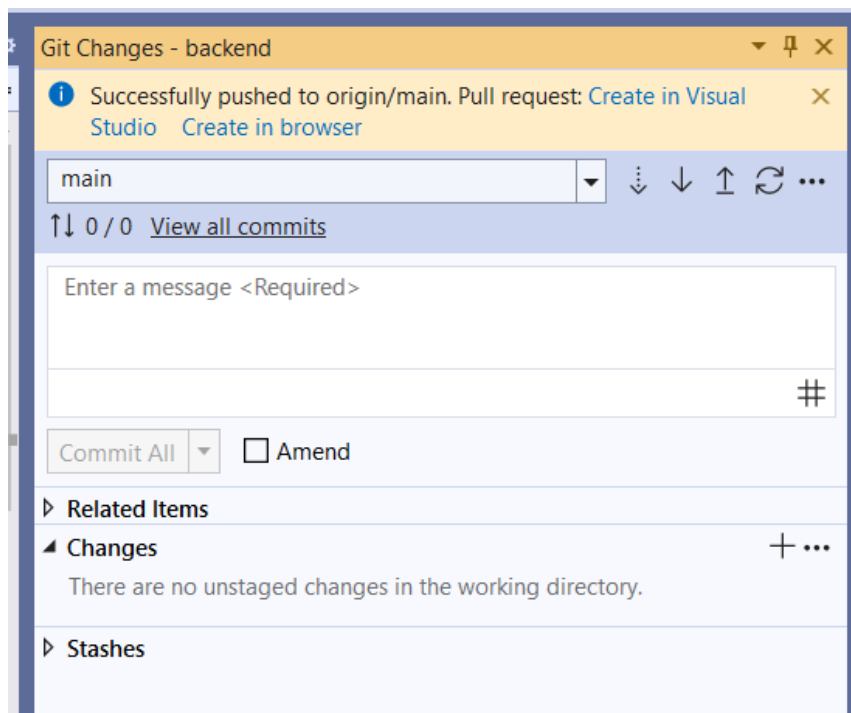
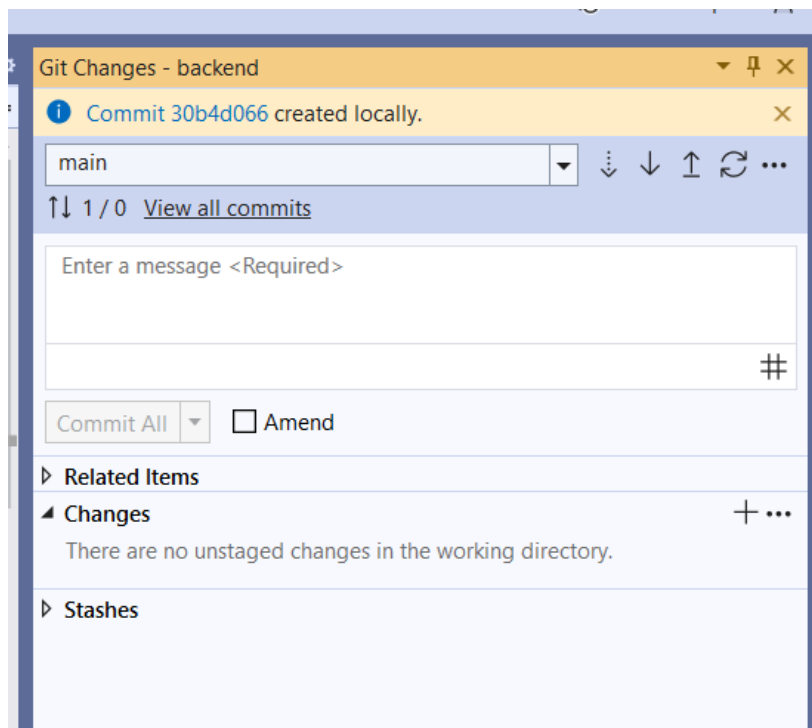
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet test
Restauración completada (0,6s)
ReservasApi realizado correctamente (2,0s) → ReservasApi\bin\Debug\net9.0\ReservasApi.dll
ReservasApi.Tests realizado correctamente (5,3s) → ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445ala (64-bit .NET 9.0.11)
[xUnit.net 00:00:00.06] Discovering: ReservasApi.Tests
[xUnit.net 00:00:00.09] Discovered: ReservasApi.Tests
[xUnit.net 00:00:00.09] Starting: ReservasApi.Tests
[xUnit.net 00:00:00.71] ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas [FAIL]
[xUnit.net 00:00:00.71] Assert.Equal() Failure: Values differ
[xUnit.net 00:00:00.71] Expected: 2
[xUnit.net 00:00:00.71] Actual: 0
[xUnit.net 00:00:00.71] Stack Trace:
[xUnit.net 00:00:00.72] C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasControllerTests.cs(48,0): at ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas()
[xUnit.net 00:00:00.72] --- End of stack trace from previous location ---
[xUnit.net 00:00:00.72] Finished: ReservasApi.Tests
ReservasApi.Tests prueba error con 1 errores (2,8s)
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasControllerTests.cs(48): error TESTERROR:
ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas (581ms): Mensaje de error: Assert.Equal() Failure: Values differ
Expected: 2
Actual: 0
Seguimiento de la pila:
at ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas() in C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\Reservas
Api.Tests\ReservasControllerTests.cs:line 48
--- End of stack trace from previous location ---

Resumen de pruebas: total: 2; con errores: 1; correcto: 1; omitido: 0; duración: 2,7 s
Compilación error con 1 errores en 11,1s

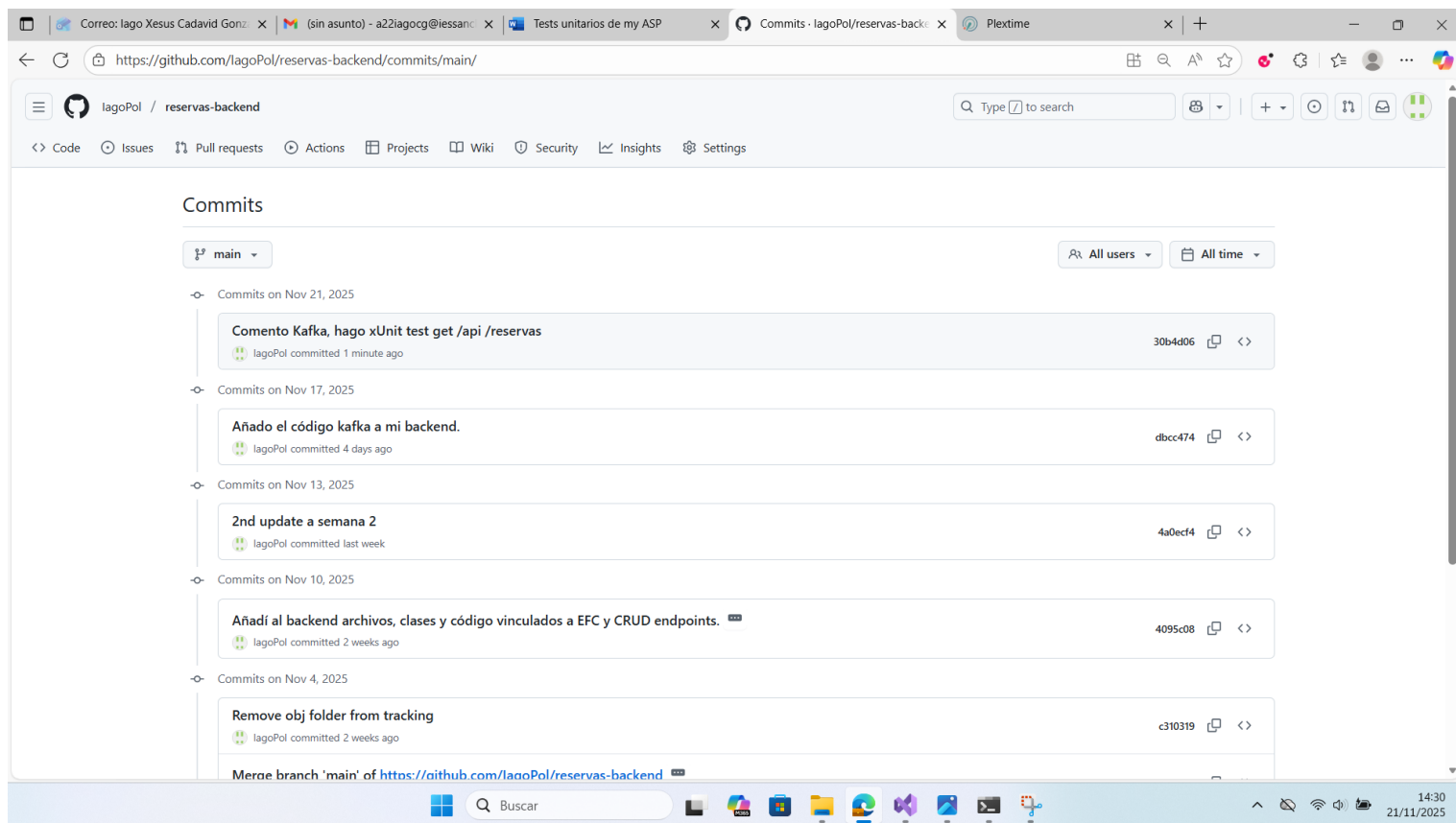
```

Chatgpt no dio con la tecla de por qué en “actual” sale 0 en vez de 2.





NEXT PAGE



27 nov I still get:

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet test
Restauración completada (0,6s)
ReservasApi realizado correctamente (0,4s) → ReservasApi\bin\Debug\net9.0\ReservasApi.dll
ReservasApi.Tests realizado correctamente (0,1s) → ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.11)
[xUnit.net 00:00:00.06] Discovering: ReservasApi.Tests
[xUnit.net 00:00:00.09] Discovered: ReservasApi.Tests
[xUnit.net 00:00:00.09] Starting: ReservasApi.Tests
[xUnit.net 00:00:00.66] ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas [FAIL]
[xUnit.net 00:00:00.66] Assert.Equal() Failure: Values differ
[xUnit.net 00:00:00.66] Expected: 2
[xUnit.net 00:00:00.66] Actual: 0
[xUnit.net 00:00:00.66] Stack Trace:
[xUnit.net 00:00:00.66] C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasControllerTests.cs(48,0): at ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas()
[xUnit.net 00:00:00.66] --- End of stack trace from previous location ---
[xUnit.net 00:00:00.66] Finished: ReservasApi.Tests
ReservasApi.Tests pruebaerror con 1 errores (1,4s)
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasControllerTests.cs(48): error TESTERROR:
ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas (526ms): Mensaje de error: Assert.Equal() Failure: Values differ
Expected: 2
Actual: 0
Seguimiento de la pila:
at ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas() in C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasControllerTests.cs:line 48
--- End of stack trace from previous location ---

Resumen de pruebas: total: 2; con errores: 1; correcto: 1; omitido: 0; duración: 1,4 s
Compilación error con 1 errores en 2,9s
```

Con este código tampoco supera el test:

```
.cs ▸ X ReservasControllerTests.cs ▸ X ReservasController.cs
rvasApi.Tests ▾ ReservasApi.Tests.ReservasControllerTests ▾ GetReservas_RetornaAllReservas()
3 | // using System.Text;
4 |
5 | namespace ReservasApi.Tests
6 | {
7 |     0 references
8 |     public class ReservasControllerTests
9 |     {
10 |         // Helper method to create a new in-memory DbContext
11 |         // Set up In-Memory AppDbContext for tests
12 |         // Each test should use a fresh in-memory database to avoid state sharing between tests.
13 |         1 reference
14 |         private AppDbContext GetInMemoryDbContext() // string dbName
15 |         {
16 |             var opciones = new DbContextOptionsBuilder<AppDbContext>()
17 |                 .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString()) // dbName can be unique per test ("TestDb1", "TestDb2") to isolate them.
18 |                 .Options;
19 |             return new AppDbContext(opciones);
20 |         } // todo este metodo privado aparece greyed out while not yet used anywhere
21 |
22 |         // Test GET all reservas
23 |         [Fact] // each test method must have the [Fact] attribute for xUnit to recognize it.
24 |         0 references
25 |         public async Task GetReservas_RetornaAllReservas() // the test method must be inside the class that is marked public class ReservasControllerTests.
26 |         {
27 |             // Arrange:
28 |             using var contexto = GetInMemoryDbContext(); // "TestDb1" // este método retorna un 'AppDbContext', que es el tipo de dato del parámetro de ReservasController constructor.
29 |
30 |             contexto.Reservas.Add(new Reserva { ReservaId = 1, Dni = "12349876F", InstalacionId = 1, NumeroAsistentes = 2, FechaHora = new DateTime(2025, 12, 15, 10, 00, 00) }); // según chatgpt no me hace falta poner FechaHora
31 |
32 |             contexto.Reservas.Add(new Reserva { ReservaId = 2, Dni = "67895432C", InstalacionId = 2, NumeroAsistentes = 3, FechaHora = new DateTime(2025, 12, 18, 15, 30, 00) });
33 |
34 |             contexto.SaveChanges();
35 |
36 |             // instancio ReservasController pasándole el 'contexto' que justo antes inicialicé con el retorno de GetInMemoryDbContext.
37 |             var controlador = new ReservasController(contexto); // 'contexto' es de tipo AppDbContext, que es el tipo del param del constructor de ReservasController
38 |
39 |             // Act:
40 |             var resultado = await controlador.GetReservas(); // GetReservas lo tengo definido en ReservasController.cs
41 |
42 |             // Assert adaptado al mi código de GetReservas() :
43 |             var okResultado = Assert.IsType<OkObjectResult>(resultado.Result); // idem assert genérico
44 |             var reservas = Assert.IsAssignableFrom<List<ReservaLeerDto>>(okResultado.Value); // cambio Reserva (assert genérico) por ReservaLeerDto
45 |             Assert.Equal(2, reservas.Count); // idem assert genérico.
46 |             /*
47 |             // Assert genérico (used when a controller returns entities directly):
48 |             var okResultado = Assert.IsType<OkObjectResult>(resultado.Result);
49 |             var reservas = Assert.IsAssignableFrom<List<Reserva>>(okResultado.Value);
50 |             Assert.Equal(2, reservas.Count);
51 |             */
52 |         } // final de [FACT] GetReservas_RetornaAllReservas()
53 |
54 |         // You can add more [Fact] methods here for POST, GET by ID, PUT, DELETE
55 |     }
56 | }
```

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet test
Restauración completada (0,8s)
ReservasApi realizado correctamente (0,4s) → ReservasApi\bin\Debug\net9.0\ReservasApi.dll
ReservasApi.Tests realizado correctamente (2,7s) → ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445ala (64-bit .NET 9.0.11)
[xUnit.net 00:00:00.08] Discovering: ReservasApi.Tests
[xUnit.net 00:00:00.11] Discovered: ReservasApi.Tests
[xUnit.net 00:00:00.11] Starting: ReservasApi.Tests
[xUnit.net 00:00:00.83] ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas [FAIL]
[xUnit.net 00:00:00.83] Assert.Equal() Failure: Values differ
[xUnit.net 00:00:00.83] Expected: 2
[xUnit.net 00:00:00.83] Actual: 0
[xUnit.net 00:00:00.83] Stack Trace:
[xUnit.net 00:00:00.83] C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasControllerTests.cs(52,0): at ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas()
[xUnit.net 00:00:00.83] --- End of stack trace from previous location ---
[xUnit.net 00:00:00.83] Finished: ReservasApi.Tests
ReservasApi.Tests pruebaerror con 1 errores (1,6s)
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasControllerTests.cs(52): error TESTERROR:
ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas (673ms): Mensaje de error: Assert.Equal() Failure: Values differ
Expected: 2
Actual: 0
Seguimiento de la pila:
at ReservasApi.Tests.ReservasControllerTests.GetReservas_RetornaAllReservas() in C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi.Tests\ReservasControllerTests.cs:line 52
--- End of stack trace from previous location ---

Resumen de pruebas: total: 2; con errores: 1; correcto: 1; omitido: 0; duración: 1,6 s
Compilación error con 1 errores en 6,0s
```

Vamos a probar a crear the related Usuario e Instalacion objects in the in-memory database before adding the Reserva. EN EFECTO FUE LA SOLUCIÓN.

NEXT PAGE

```

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet test
Restauración completada (0,9s)
ReservasApi realizado correctamente (0,5s) → ReservasApi\bin\Debug\net9.0\ReservasApi.dll
ReservasApi.Tests realizado correctamente (2,8s) → ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.11)
[xUnit.net 00:00:00.07] Discovering: ReservasApi.Tests
[xUnit.net 00:00:00.09] Discovered: ReservasApi.Tests
[xUnit.net 00:00:00.10] Starting: ReservasApi.Tests
[xUnit.net 00:00:00.75] Finished: ReservasApi.Tests
ReservasApi.Tests prueba realizado correctamente (1,5s)

Resumen de pruebas: total: 2; con errores: 0; correcto: 2; omitido: 0; duración: 1,5 s
Compilación realizado correctamente en 6,0s

```

Este fue el resultado del test, el código que lo logra es éste otro:

```

Reserva.cs  ReservasControllerTests.cs  Usuario.cs  ReservasController.cs
ReservasApi.Tests
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ReservasApi.Controllers;
using ReservasApi.Data;
using ReservasApi.Modelos;
using ReservasApi.Modelos.DTO;
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Xunit;

// using System.Linq;
// using System.Text;

```

F

```

15 namespace ReservasApi.Tests
16 {
17     0 references
18     public class ReservasControllerTests
19     {
20         // Helper method to create a new in-memory DbContext
21         // Set up In-Memory AppDbContext for tests
22         // Each test should use a fresh in-memory database to avoid state sharing between tests.
23         1 reference
24         private AppDbContext GetInMemoryDbContext() // string dbName
25         {
26             var opciones = new DbContextOptionsBuilder<AppDbContext>()
27                 .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString()) // dbName can be unique per test ("TestDb1", "TestDb2") to isolate them.
28                 .Options;
29             return new AppDbContext(opciones);
30         } // todo este metodo privado aparece greyed out while not yet used anywhere
31
32         // Test GET all reservas
33         [Fact] // each test method must have the [Fact] attribute for xUnit to recognize it.
34         0 references
35         public async Task GetReservas_ReturnaAllReservas() // the test method must be inside the class that is marked public class ReservasControllerTests.
36         {
37             // Arrange:
38             using var contexto = GetInMemoryDbContext(); // "TestDb1" // este método retorna un 'AppDbContext', que es el tipo de dato del parámetro de ReservasController construct
39
40             // Seed Usuarios:
41             // creo the related Usuario e Instalacion objects in the in-memory database before adding the Reserva:
42             var usuario1 = new Usuario { Dni = "12349876F", NombreCompleto = "Martin Gomez", Email = "mgomez@gmail.com", Telefono = "988765432", Domicilio = "Calle Falsa 123", FechaN
43             var usuario2 = new Usuario { Dni = "67895432C", NombreCompleto = "Jane Smith", Email = "jsmith@gmail.com", Telefono = "981735230", Domicilio = "Calle Real 321", FechaN
44             contexto.Usuarios.AddRange(usuario1, usuario2);
45
46             // Seed Instalaciones:
47             var instalacion1 = new Instalacion { InstalacionId = 1, Localizacion = "Pabellon A", CapacidadMaxima = 50, PrecioHora = 20.0, Tipo = true };
48             var instalacion2 = new Instalacion { InstalacionId = 2, Localizacion = "Pabellon B", CapacidadMaxima = 30, PrecioHora = 15.0, Tipo = false};
49             contexto.Instalaciones.AddRange(instalacion1, instalacion2);

```



```

48 // Seed Reservas:
49 contexto.Reservas.AddRange(
50     new Reserva
51     {
52         ReservaId = 1,
53         Dni = usuario1.Dni,
54         InstalacionId = instalacion1.InstalacionId,
55         NumeroAsistentes = 2,
56         FechaHora = new DateTime(2025, 12, 15, 10, 0, 0)
57     },
58     new Reserva
59     {
60         ReservaId = 2,
61         Dni = usuario2.Dni,
62         InstalacionId = instalacion2.InstalacionId,
63         NumeroAsistentes = 3,
64         FechaHora = new DateTime(2025, 12, 18, 15, 30, 0)
65     }
66 );
67 contexto.Reservas.Add(new Reserva { ReservaId = 1, Dni = "12349876F", InstalacionId = 1, NumeroAsistentes = 2, FechaHora = new DateTime(2025, 12, 15, 10, 00, 00) });
68 // contexto.Reservas.Add(new Reserva { ReservaId = 2, Dni = "67895432C", InstalacionId = 2, NumeroAsistentes = 3, FechaHora = new DateTime(2025, 12, 18, 15, 30, 00) });
69
70 contexto.SaveChanges();
71
72 // instancio ReservasController pasándole el 'contexto' que justo antes inicialicé con el retorno de GetInMemoryDbContext.
73 var controlador = new ReservasController(contexto); // 'contexto' es de tipo AppDbContext, que es el tipo del param del constructor de ReservasController
74
75 // Act:
76 var resultado = await controlador.GetReservas(); // GetReservas lo tengo definido en ReservasController.cs
77
78 // Assert adaptado al mi código de GetReservas() :
79 var okResultado = Assert.IsType<OkObjectResult>(resultado.Result); // idem assert genérico
80 var reservas = Assert.IsAssignableFrom<List<ReservaLeerDto>>(okResultado.Value); // cambio Reserva (assert genérico) por ReservaLeerDto
81 Assert.Equal(2, reservas.Count); // idem assert genérico.

```

```

82 /*
83 // Assert genérico (used when a controller returns entities directly):
84 var okResultado = Assert.IsType<OkObjectResult>(resultado.Result);
85 var reservas = Assert.IsAssignableFrom<List<Reserva>>(okResultado.Value);
86 Assert.Equal(2, reservas.Count);
87 */
88 } // final de [FACT] GetReservas_ReturnaAllReservas()
89
90 // You can add more [Fact] methods here for POST, GET by ID, PUT, DELETE
91 }
92 }
93

```

Habiendo logrado ese test, podemos afirmar esto:

You've basically got a fully working pattern for unit-testing your controller with EF Core In-Memory Database:

- Each test gets a **fresh in-memory database**.
- All **required related entities** (`Usuario` , `Instalacion`) are seeded.
- Your controller methods work **exactly like in production**, without touching Kafka or SQL Server.

Next steps (optional, but recommended):

1. Add similar tests for:
 - `GET /api/reservas/{id}`
 - `POST /api/reservas`
 - `PUT /api/reservas/{id}`
 - `DELETE /api/reservas/{id}`
2. Consider **factoring out repeated seed logic** into helper methods to keep tests DRY.

Lo siguiente que quiero hacer es codificar un test unitario por cada GET all rows (uno para cada tabla).

GET /api /usuarios (está en **UsuariosController.cs**):

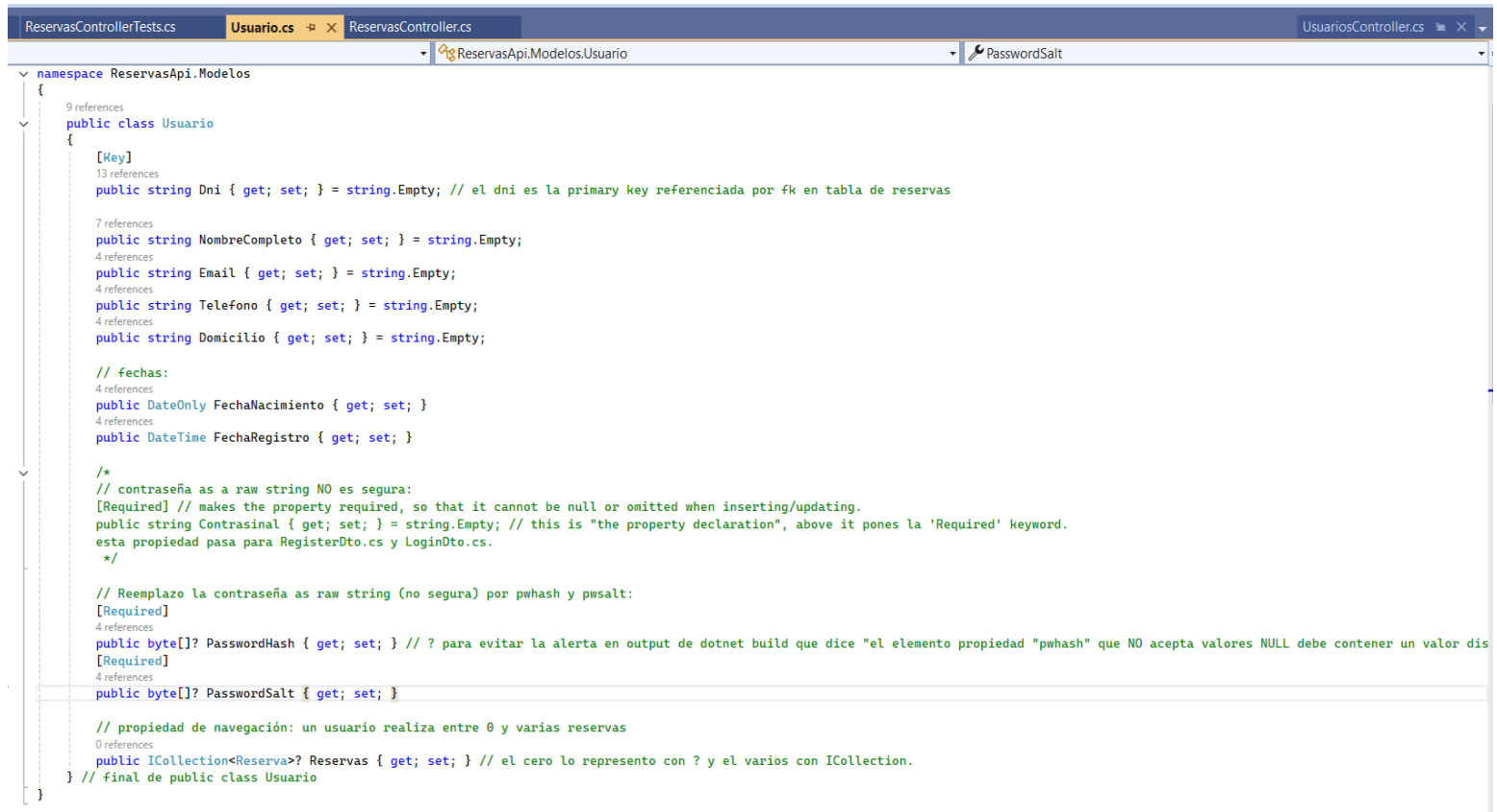
Debido a que mi GET /api /usuarios **returns Usuario entities directly** instead of DTOs, this test should be much simpler than the get api reservas one.

Recordemos que mi get api usuarios es éste:

```
// GET: api/usuarios
// GetUsuarios tiene por propósito retornar una lista de todos los usuarios from the database.
[HttpGet]
0 references
public async Task<ActionResult<IEnumerable<Usuario>>> GetUsuarios()
{
    var usuarios = await _contexto.Usuarios.ToListAsync();
    return Ok(usuarios);
}
```

Donde ‘usuarios’ es un List de objetos Usuario.

Recordemos además el modelo de usuarios:



```
namespace ReservasApi.Modelos
{
    9 references
    public class Usuario
    {
        [Key]
        13 references
        public string Dni { get; set; } = string.Empty; // el dni es la primary key referenciada por fk en tabla de reservas

        7 references
        public string NombreCompleto { get; set; } = string.Empty;
        4 references
        public string Email { get; set; } = string.Empty;
        4 references
        public string Telefono { get; set; } = string.Empty;
        4 references
        public string Domicilio { get; set; } = string.Empty;

        // fechas:
        4 references
        public DateOnly FechaNacimiento { get; set; }
        4 references
        public DateTime FechaRegistro { get; set; }

        /*
        // contraseña as a raw string NO es segura:
        [Required] // makes the property required, so that it cannot be null or omitted when inserting/Updating.
        public string Contraseñal { get; set; } = string.Empty; // this is "the property declaration", above it pones la 'Required' keyword.
        esta propiedad pasa para RegisterDto.cs y LoginDto.cs.
        */

        // Reemplazo la contraseña as raw string (no segura) por pwhash y pwsalt:
        [Required]
        4 references
        public byte[]? PasswordHash { get; set; } // ? para evitar la alerta en output de dotnet build que dice "el elemento propiedad "pwhash" que NO acepta valores NULL debe contener un valor dis
        [Required]
        4 references
        public byte[]? PasswordSalt { get; set; }

        // propiedad de navegación: un usuario realiza entre 0 y varias reservas
        0 references
        public ICollection<Reserva>? Reservas { get; set; } // el cero lo represento con ? y el varios con ICollection.
    } // final de public class Usuario
}
```

Podemos ahí observar que tabla usuarios no tiene foreign keys y su propiedad de navegación puede ser nula.

Todo eso debería simplificar el test get api usuarios con respecto al test get api reservas antes realizado.

[

A modo de comparativa, recordemos que Reserva.cs tenía un par de foreign keys cuyas propiedades de navegación no deberían ser nulas (digo “no deberían” en vez de “no pueden” porque uso el null-forgiving operator = null!):

```
Reserva.cs  ReservasControllerTests.cs  Usuario.cs  ReservasController.cs  UsuariosController.cs
ReservasApi
namespace ReservasApi.Modelos
{
    4
    5
    6 public class Reserva
    7 {
    8     // para cada campo empleo la sintaxis sintética propia de c sharp para getter y setter, fusiona varias líneas de código en una sola
    9     public int ReservaId { get; set; } // pk de tabla de reservas. No puse encima [Key] porque le llamé ReservaId; in EFC, you don't need the [Key] attribute if your property follows EF Core's naming conventions -a
10     // EF Core automatically recognizes ReservaId (or Id) as a primary key, so no extra attributes are required unless you prefer to be explicit (por ejemplo [Key]).
11     public int NumeroAsistentes { get; set; } // cuánta gente asistirá al espacio reservado
12     public DateTime FechaHora { get; set; } // fecha y hora de asistencia al espacio reserva; la fecha de inicio.
13     // me falta poner FechaFin o duración de la reserva.
14     // FK que referencia a PK de tabla de usuarios oasea al dni:
15     [Required] // esto exige poner: using System.ComponentModel.DataAnnotations;
16     [ForeignKey("Usuario")] // esto exige poner: using System.ComponentModel.DataAnnotations.Schema;
17     public string Dni { get; set; } = string.Empty; // FK -> pk (dni) de tabla de usuarios registrados
18
19     // Propiedad de navegación:
20     public Usuario Usuario { get; set; } = null!; // una reserva siempre es realizada por un solo usuario ni más ni menos (por ende NO pongo ?).
21     // la exclamación en null! is the null-forgiving operator. It means "yes, compiler, I promise this property will not actually be null at runtime -even though I am initializing it with null for now". It silences
22     // usamos null! porque EFCore does not actually assign "it" (a value, diría yo) right away when creating a new entity (al crear un nuevo objeto de la clase Usuario, diría yo)
23
24     // fk que referencia a pk de tabla de instalaciones
25     [Required]
26     [ForeignKey("Instalacion")] // con esto tengo entendido que es suficiente para que EFC sepa que esta fk apunta a la pk de tabla instalaciones.
27     public int InstalacionId { get; set; } // FK -> pk de tabla de instalaciones
28
29     // Propiedad de navegación:
30     public Instalacion Instalacion { get; set; } = null!; // una reserva siempre lo es sobre una instalación ni más ni menos (por ende no pongo ?).
31     // la exclamación en null! is the null-forgiving operator. It means "yes, compiler, I promise this property will not actually be null at runtime -even though I am initializing it with null for now". It silences
32
33     // Relación (1:1) con Pago, propiedad de navegación:
34     public Pago? Pago { get; set; } // Cada reserva tiene un pago asociado (no más) y de momento voy dejar abierta la posibilidad de que exista una reserva impagada, lo cual trato de transmitir con ?.
35
36     // Relación (1:1) con Cancelacion, propiedad de navegación:
37     public Cancelacion? Cancelacion { get; set; } // Cada reserva es cancelada por como máximo una cancelación (y evidentemente una reserva podrá no haber sido cancelada oasea podrá estar asociada a cero cancelaci
38     // en relación a property names, podría ser necesaria una configuración en ApplicationDbContext.
39 }
40
```

]

Por si ayuda/interesa,

El test GET /api/reservas con xUnit que me fallaba (expected: 2, actual: 0) ahora da éxito tras hacerle 3 cambios (guiones 2 y 3 constituyen el 2ndo cambio):

- En las dos new Reserva antes no les ponía valor para el campo de fecha de inicio de la reserva, ahora sí se lo pongo. Este campo en mi modelo de la tabla de reservas no lleva [Required], pero tampoco ? junto al tipo de dato DateTime.
- private ApplicationDbContext GetInMemoryDbContext() // le quité string dbName como parámetro al GetInMemoryDbContext()
- .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString()) // antes tenía dbName en lugar de Guid.etc.
- además de crear dos new Reserva, ahora también creo dos usuarios y dos instalaciones, puesto que ambos son propiedades de navegación y fks requeridas en mi modelo de la tabla de reservas.

Buena tarde 🤗

Todos esos cambios son en el archivo ReservasControllerTests.cs, que está dentro del proyecto de los tests xUnit.

Hoy

Por lo que veo es más simple codificar el test unitario xUnit cuando tu GET del controller retorna directamente la entidad de la tabla (cosa que por ejemplo en mi caso pasa con GET /api/usuarios) que cuando retorna DTOs (que en mi caso pasa con get /api/reservas). También es más simple cuando la tabla que estás testeando no tiene claves foráneas requeridas (en mi tabla de usuarios no las hay VS en mi tabla de reservas hay dos).

Regarding "// Reuse your existing in-memory helper or copy this:

```
private AppDbContext GetInMemoryDbContext()
{
    var options = new DbContextOptionsBuilder<AppDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .Options;
    return new AppDbContext(options);
}
```

I notice that is exactly the same we have in ReservasControllerTests.cs, so how can I reuse it in UsuariosControllerTests instead of copying it again ?

Great question — and an important one, because **you should never duplicate test utilities** when you can reuse them cleanly.

You have **three good options**, and I will show you each one in order from best practice → simplest.

✅ Option 1 (Best Practice): Create a shared Test Helper class

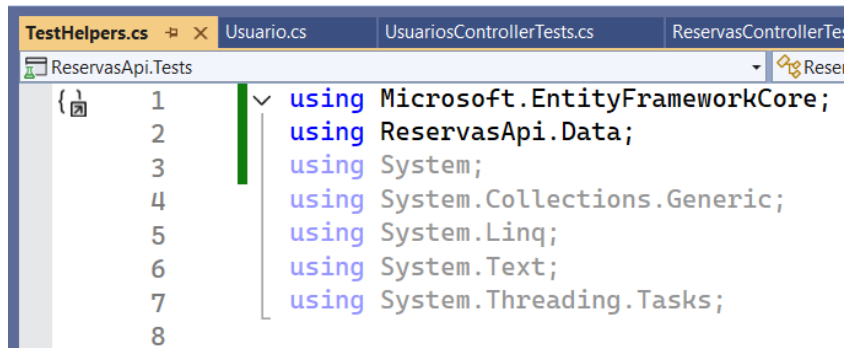
Create a new file:

 TestHelpers.cs (inside the ReservasApi.Tests project)

```
9 namespace ReservasApi.Tests
10 {
11     /*
12     // static because: it doesn't store any data; it only provides utility methods;
13     it's ideal for pure utility functions that do not depend on object instances;
14     you will never write sth like var helpers = new TestHelpers();
15     static PREVENTS INSTANTIATION and (thus) AVOIDS UNNECESARY OBJECT CREATION;
16     utility classes in C# are conventionally static, fex. Math.Sin(), Path.Combine() or File.ReadAllText();
17     My test helpers belong to this category.
18     */
19     0 references
20     public static class TestHelpers
21     {
22         0 references
23         public static AppDbContext GetInMemoryDbContext() // lo mismo que inicial y directamente yo antes ponía en ReservasControll
24         {
25             var opciones = new DbContextOptionsBuilder<AppDbContext>()
26                 .UseInMemoryDatabase(Guid.NewGuid().ToString())
27                 .Options;
28             return new AppDbContext(opciones);
29         } // final de la función pública y estática sin parámetros 'GetInMemoryDbContext' que retorna AppDbContext.
30     } // final de la clase pública y estática TestHelpers.
31 } // end of namespace
32
```

Final del comentario línea 21: “en ReservasControllerTests.cs, si bien en este caso ponía private en lugar de public static.”

Los using de TestHelpers.cs son estos:



```
TestHelpers.cs  Usuario.cs  UsuariosControllerTests.cs  ReservasControllerTests.cs
ReservasApi.Tests
1  using Microsoft.EntityFrameworkCore;
2  using ReservasApi.Data;
3  using System;
4  using System.Collections.Generic;
5  using System.Linq;
6  using System.Text;
7  using System.Threading.Tasks;
8
```

[

Sobre el tema de por qué static tanto la clase como la función que contiene en vez de solo static una de ellas, en especial con respecto a cuándo se puede hacer lo siguiente:

var db = TestHelpers.GetInMemoryDbContext();

```
I'm wondering whether you could write using var db =
TestHelpers.GetInMemoryDbContext(); if the code in TestHelpers.cs
were the following one:

using Microsoft.EntityFrameworkCore;
using ReservasApi.Data;

namespace ReservasApi.Tests
{
    public class TestHelpers // noticed I erase static here, but not in
    GetInMemoryDbContext()
    {
        public static AppDbContext GetInMemoryDbContext() // I keep
        static here.
        {
            var options = new DbContextOptionsBuilder<AppDbContext>()
                .UseInMemoryDatabase(Guid.NewGuid().ToString())
                .Options;

            return new AppDbContext(options);
        }
    }
}
```

Yes — you absolutely *can* still write:

```
csharp
using var db = TestHelpers.GetInMemoryDbContext();
```

even if the *class itself* is **not static**, as long as the **method** is **static**.

Esto es lo que yo indeed me esperaba.

]

✔ Why this still works

In C#, you can always call a **static method** directly on the class name:

```
csharp
TestHelpers.GetInMemoryDbContext();
```

This works **whether the class is static or non-static**.

So both of these are valid:

◆ Static class + static method

```
csharp
public static class TestHelpers
{
    public static ApplicationDbContext GetInMemoryDbContext() { ... }
}
```

◆ Non-static class + static method

```
csharp
public class TestHelpers
{
    public static ApplicationDbContext GetInMemoryDbContext() { ... }
}
```

Both allow:

```
csharp
var db = TestHelpers.GetInMemoryDbContext();
```



✔ So what changes if the class is NOT static?

Nothing in usage.

You can still call the static method the same way.

The only difference:

Because the class is not static, someone *could* do this:

```
csharp
var helper = new TestHelpers();
```

Even though they don't need to.

Static classes prevent that.

✔ Recommendation

✔ **Best practice:** Make the class `static` if:

- all methods are static
- the class holds no instance data
- you never need to instantiate it

✗ There is no functional problem if you don't make the class static

✔ It just makes intent clearer and avoids unnecessary instantiation.

NEXT PAGE

And what if the class were static but its function weren't, could you still write `var db = TestHelpers.GetInMemoryDbContext();` or not ?

No — you cannot.

If the class is `static` but the method is **not** static, then this:

csharp

 Copy code

```
var db = TestHelpers.GetInMemoryDbContext();
```

✗ WILL NOT COMPILE.

✓ Why?

A `static` class in C#:

- cannot be instantiated
- cannot contain instance members
- cannot contain non-static methods

So this code is **illegal**:

csharp

 Copy code

```
public static class TestHelpers
{
    public ApplicationDbContext GetInMemoryDbContext() // ✗ Not allowed
    {
        ...
    }
}
```

The compiler will give:

`"Static classes cannot contain instance members."`

Según eso, un non-static method is an instance member.

Haciendo uso de esa nueva clase `TestHelpers`, el `UsuariosControllerTests.cs` quedaría con este código:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using ReservasApi.Controladores;
using ReservasApi.Data;
using ReservasApi.Modelos;
using Xunit;

namespace ReservasApi.Tests
{
    public class UsuariosControllerTests
    {
        [Fact]
        public async Task GetUsuarios_ReturnsAllUsuarios()
        {
            // Arrange
            using var context = TestHelpers.GetInMemoryDbContext();

            context.Usuarios.AddRange(
                new Usuario
                {
                    Dni = "12345678A",
                    NombreCompleto = "John Doe",
                    Email = "john@example.com",
                    Telefono = "111222333",
                    Domicilio = "Calle Uno 1",
                    FechaNacimiento = new DateOnly(1990, 1, 1),
                    FechaRegistro = DateTime.UtcNow,
                    PasswordHash = new byte[] { 1, 2, 3 },
                    PasswordSalt = new byte[] { 4, 5, 6 }
                },
            );
        }
    }
}
```

NEXT PAGE

```

new Usuario
{
    Dni = "876543218",
    NombreCompleto = "Jane Smith",
    Email = "jane@example.com",
    Telefono = "444555666",
    Domicilio = "Calle Dos 2",
    FechaNacimiento = new DateOnly(1985, 6, 15),
    FechaRegistro = DateTime.UtcNow,
    PasswordHash = new byte[] { 7, 8, 9 },
    PasswordSalt = new byte[] { 10, 11, 12 }
};

await context.SaveChangesAsync();

var controller = new UsuariosController(context);

// Act
var result = await controller.GetUsuarios();

// Assert
var okResult = Assert.IsType<OkObjectResult>(result.Result);
var usuarios = Assert.IsAssignableFrom<IEnumerable<Usuario>>(okResult.Value);

Assert.Equal(2, usuarios.Count());

Assert.Contains(usuarios, u => u.Dni == "12345678A");
Assert.Contains(usuarios, u => u.Dni == "876543218");
}
}
}

```

[

Comparemos ese código con el código que no usa la clase TestHelpers sino que directamente pone el código de dicha clase. Dicho código (el que no usa la clase TestHelpers) lo muestro a continuación:

NEXT PAGE

```

using ReservasApi.Controllers;
using ReservasApi.Modelos;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace ReservasApi.Tests
{
    public class UsuariosControllerTests
    {
        // Reuse your existing in-memory helper or copy this
        private AppDbContext GetInMemoryDbContext()
        {
            var options = new DbContextOptionsBuilder<AppDbContext>()
                .UseInMemoryDatabase(Guid.NewGuid().ToString())
                .Options;
            return new AppDbContext(options);
        }
    }
}

```

```

[Fact]
public async Task GetUsuarios_ReturnaTodosUsuarios()
{
    // Arrange
    using var contexto = GetInMemoryDbContext();

    contexto.Usuarios.Add(new Usuario
    {
        Dni = "193709870",
        NombreCompleto = "Manuel Molano",
        Email = "molano@gmail.com",
        Telefono = "635786203",
        Domicilio = "Calle Felicidad 9, Vigo",
        FechaNacimiento = new DateOnly(2000, 5, 24),
        FechaRegistro = new DateTime(2025, 11, 27, 11, 30, 55),

        // Required fields → minimal dummy values
        PasswordHash = new byte[] { 1, 2, 3 },
        PasswordSalt = new byte[] { 4, 5, 6 }
    });
}

```

```

contexto.Usuarios.Add(new Usuario
{
    Dni = "98761234Z",
    NombreCompleto = "Laura Gomez",
    Email = "laugom@gmail.com",
    Telefono = "981856743",
    Domicilio = "Calle Felicidad 9, Madrid",
    FechaNacimiento = new DateOnly(1990, 5, 10),
    FechaRegistro = new DateTime(2025, 11, 12, 14, 51, 43),

    PasswordHash = new byte[] { 7, 8, 9 },
    PasswordSalt = new byte[] { 10, 11, 12 }
});

contexto.SaveChanges();

var controller = new UsuariosController(contexto);

// Act
var resultado = await controller.GetUsuarios();

// Assert
var okResultado = Assert.IsType<OkObjectResult>(resultado.Result);
var usuarios = Assert.IsAssignableFrom<List<Usuario>>(okResultado.Value);

Assert.Equal(2, usuarios.Count);
}
}
}

```

Ask ChatGPT

Ahí donde pone Ask ChatGPT

lo que realmente pone es // Act.

Diferencias observadas:

1. using var context = **TestHelpers**.GetInMemoryDbContext(); // antes no ponía TestHelpers (obviamente)

2. Antes después de añadir los dos usuarios, ponía contexto.SaveChanges(); VS ahora pone:

await context.SaveChanges**Async**();

Y no creo que la razón se porque antes añadía los usuarios con contexto.Usuarios.add x 2 VS ahora con un solo contexto.Usuarios.AddRange.

3. var usuarios = Assert.IsAssignableFrom<**IEnumerable**<Uusuario>>(okResult.Value); // **List** antes

✓ 1. Why now `await context.SaveChangesAsync();` instead of `context.SaveChanges()` ?

Short answer

Because **your controller is async**, and your test should follow the same async pattern.
It's considered best practice in modern .NET.

Long answer

In your earlier code, you were adding users like this:

```
csharp

context.Usuarios.AddRange(...);
context.SaveChanges(); // sync version
```

This works — but your controller action is asynchronous:

```
csharp

public async Task<ActionResult<IEnumerable<Usuario>>> GetUsuarios()
```

So, the test should ideally use:

```
csharp

await context.SaveChangesAsync();
```

Lo de “your controller action is asynchronous” lo saca de aquí (en mi UsuariosController.cs code):

```
// GET: api/usuarios
// GetUsuarios tiene por propósito retornar una lista de todos los usuarios from the database.
[HttpGet]
0 references
public async Task<ActionResult<IEnumerable<Usuario>>> GetUsuarios()
```

Tmb añadir que siendo exactos, el earlier code como ya dije no usaba `AddRange` x 1 sino `Add` x 2, pero bueno esto no ha de ser relevante.

Sobre la diferencia de `var usuarios = Assert.IsAssignableFrom<IEnumerable<Usuario>>(okResult.Value);` // antes en vez de `IEnumerable` era `List`, en primer lugar aclarar esto:

NEXT PAGE

✓ Which method has "IEnumerable" in the signature?

I'm referring to your controller action:

```
csharp

[HttpGet]
public async Task<ActionResult<IEnumerable<Usuario>>> GetUsuarios()
```

The return type of this method is:

```
swift

ActionResult<IEnumerable<Usuario>>
```

That means:

- the controller promises to return a **collection of Usuario**,
- but it does **not** guarantee which *specific* collection type (List, array, etc.)

Even though inside the method you write:

```
csharp

var usuarios = await _contexto.Usuarios.ToListAsync(); // this is a List<Usua
return Ok(usuarios);
```

The return type of the method itself is more general:

```
php-template

IEnumerable<Usuario>
```

"Your controller action" = get api usuarios en

UsuariosController.cs.

NEXT PAGE justificación del cambio:

✓ 2. Why now `IEnumerable<Usuario>` instead of `List<Usuario>`?

Short answer

Because ASP.NET Core returns a `List`, but your method signature returns `IEnumerable`, so using `IEnumerable<Usuario>` is more general and safer.

Your action returns:

```
csharp
return Ok(usuario); // usuarios is a List<Usuario>
```

`OkObjectResult.Value` is typed as an object, so your test must cast it back to something.

Initially, I wrote:

```
csharp
var usuarios = Assert.IsType<List<Usuario>>(okResult.Value);
```

This works, but it's slightly stricter than needed.

Why I changed it to `IEnumerable<Usuario>`:

```
csharp
var usuarios = Assert.IsAssignableFrom<IEnumerable<Usuario>>(okResult.Value);
```

- `IsAssignableFrom<T>` allows:
 - `List<T>`
 - `T[]`
 - Any collection implementing `IEnumerable<T>`
- Future-proof if you later return `IQueryable<Usuario>` or `Usuario[]`.

- ✓ More flexible
- ✓ Still correct
- ✓ Same assertions work (`Count()` , `Contains`)

Cuando dice “because ASP.NET Core returns a List parece ser que se refiere a la línea:

`var usuarios = await _contexto.Usuarios.ToListAsync();` que está dentro del get api usuarios en `UsuariosController.cs`”.

"Because ASP.NET Core returns a List" sounds abstract to me, I mean what is the real subject of "returns a List" there ?

Ah, I see — you want to pin down exactly what "returns a List" means in that sentence. Let's break it down carefully.

1 The method in question

csharp

 Copy code

```
[HttpGet]
public async Task<ActionResult<IEnumerable<Usuario>>> GetUsuarios()
{
    var usuarios = await _contexto.Usuarios.ToListAsync();
    return Ok(usuarios);
}
```

2 What actually happens

- `await _contexto.Usuarios.ToListAsync()` produces a `List<Usuario>` object in memory.
 - Subject here: the variable `usuarios` is a `List<Usuario>`.
- Then we pass it to `Ok(usuarios)` :
 - `Ok(object value)` creates an `OkObjectResult` wrapping whatever object you give it.
 - So the subject of "returns a List" is the object inside the `OkObjectResult`, i.e., `usuarios`.
- ASP.NET Core doesn't change it — it just serializes it to JSON for HTTP. But inside the test, when you access `okResult.Value`, the object is still a `List<Usuario>`.

Pasemos a la justificación de la última diferencia, esta es, por qué ahora mete dos `Assert.contains` que antes no metía.

NEXT PAGE

✓ 3. Why now add two `Assert.Contains(...)` checks?

Short answer

To ensure the test validates *which* users are returned, not only *how many*.

The original test only checked:

- the result was OK
- it returned 2 users
- no validation of the **actual content**

The improved test:

csharp

 Copy code

```
Assert.Contains(usuarios, u => u.Dni == "12345678A");  
Assert.Contains(usuarios, u => u.Dni == "87654321B");
```

These additions make the test:

- stronger
- more precise
- less likely to pass when something is wrong

For example, if something inserts the wrong users, or the controller filters incorrectly, a simple count of `2` would still pass — but `Contains(dni)` would fail.

Tests should assert *behavior*, not just *shape*.

Pasa que entonces

habría que añadirlo tmb en el test de get api reservas porque si no me confundo no se lo habíamos puesto, solo testea el count de momento osease termina con `Assert.Equal(2, reservas.Count);`

]

Vamos entonces ahora a codificar ya el **UsuariosControllerTest.cs** con la segunda versión osease haciendo uso de la clase `TestHelpers.cs`, poniendo `await` y `Async` en el `context.savechanges`, poniendo `IEnumerable` en vez de `List` en el `isassignablefrom` pero de momento sin el `assert.contains x 2` para ir testeando cosita a cosita (osease primero ver si va lo del count y luego ya veremos si va con esos `assert.contains` añadidos).

UsuariosControllerTest.cs NEXT PAGE

```
1  using Microsoft.AspNetCore.Mvc;
2  using ReservasApi.Controllers;
3  using ReservasApi.Modelos;
4
5  namespace ReservasApi.Tests
6  {
7      0 references
8      public class UsuariosControllerTests
9      {
10         [Fact]
11         0 references
12         public async Task GetUsuarios_ReturnaAllUsuarios()
13         {
14             // Arrange
15             using var contexto = TestHelpers.GetInMemoryDbContext();
16
17             contexto.Usuarios.AddRange(
18                 new Usuario
19                 {
20                     Dni = "34561278C",
21                     NombreCompleto = "Marcos Gomez",
22                     Email = "mgomez@gmail.com",
23                     Telefono = "734985638",
24                     Domicilio = "Calle Ventana 4",
25                     FechaNacimiento = new DateTime(1993, 03, 18),
26                     FechaRegistro = DateTime.UtcNow,
27                     PasswordHash = new byte[] { 1, 2, 3 },
28                     PasswordSalt = new byte[] { 4, 5, 6 }
29                 },
30                 new Usuario
31                 {
32                     Dni = "40901535L",
33                     NombreCompleto = "Luis Diaz",
34                     Email = "ldiaz@gmail.com",
35                     Telefono = "884325790",
36                     Domicilio = "Calle Rotulador 4",
37                     FechaNacimiento = new DateTime(1997, 05, 20),
38                     FechaRegistro = DateTime.UtcNow,
39                     PasswordHash = new byte[] { 7, 8, 9 },
40                     PasswordSalt = new byte[] { 10, 11, 12 }
41                 }
42             ); // fin del AddRange
```

```

42      await contexto.SaveChangesAsync();
43
44      var controlador = new UsuariosController(contexto);
45
46      // Act:
47      var resultado = await controlador.GetUsuarios();
48
49      // Assert:
50      var okResultado = Assert.IsType<OkObjectResult>(resultado.Result);
51      var usuarios = Assert.IsAssignableFrom<IEnumerable<Usuario>>(okResultado.Value);
52
53      Assert.Equal(2, usuarios.Count());
54  } // fin de GetUsuarios_ReturnaAllUsuarios() method.
55
56      } // fin de la clase
57  } // end of namespace
58

```

Test da **existoso** pero con algunos avisos y siendo solo para **verificar the number of users returned, not their actual content**:

```

C:\Windows\System32\cmd.e  x  +  v
Microsoft Windows [Versión 10.0.26200.7171]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet test --filter FullyQualifiedName=UsuariosControllerTests
Restauración completada (3,4s)
ReservasApi correcto con 3 advertencias (25,1s) + ReservasApi\bin\Debug\net9.0\ReservasApi.dll
C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Controllers\AuthController.cs(79,83): warning CS8604: Posible argumento de referencia nulo para el parámetro "hashGuardado" en "bool AuthService.VerificarPwHash(string pw, byte[] hashGuardado, byte[] saltGuardado)".
C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Controllers\AuthController.cs(79,105): warning CS8604: Posible argumento de referencia nulo para el parámetro "saltGuardado" en "bool AuthService.VerificarPwHash(string pw, byte[] hashGuardado, byte[] saltGuardado)".
C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Controllers\AuthController.cs(113,40): warning CS8604: Posible argumento de referencia nulo para el parámetro "s" en "byte[] Encoding.GetBytes(string s)".
ReservasApi.Tests reutilizado correctamente (1,4s) + ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d45a1a (64-bit .NET 9.0.11)
[xUnit.net 00:00:00.06] Discovering: ReservasApi.Tests
[xUnit.net 00:00:00.09] Discovered: ReservasApi.Tests
[xUnit.net 00:00:00.09] Starting: ReservasApi.Tests
[xUnit.net 00:00:00.65] Finished: ReservasApi.Tests
ReservasApi.Tests prueba:realizado correctamente (2,5s)

Resumen de pruebas: total: 1; con errores: 0; correcto: 1; omitido: 0; duración: 2,4 s
Compilación correcto con 3 advertencias en 33,5s

C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend>

```

Añado los dos Assert.Contains y vuelvo a correr el test, entonces todavía da mejor porque no salen avisos (muestro los dos tests juntos pa comparar; debajo está el que incluye los dos Assert.Contains):

```

Microsoft Windows [Versión 10.0.26200.7171]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet test --filter FullyQualifiedName=UsuariosControllerTests
Restauración completada (3,4s)
ReservasApi correcto con 3 advertencias (25,1s) + ReservasApi\bin\Debug\net9.0\ReservasApi.dll
C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Controllers\AuthController.cs(79,83): warning CS8604: Posible argumento de referencia nulo para el parámetro "hashGuardado" en "bool AuthService.VerificarPwHash(string pw, byte[] hashGuardado, byte[] saltGuardado)".
C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Controllers\AuthController.cs(79,105): warning CS8604: Posible argumento de referencia nulo para el parámetro "saltGuardado" en "bool AuthService.VerificarPwHash(string pw, byte[] hashGuardado, byte[] saltGuardado)".
C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Controllers\AuthController.cs(113,40): warning CS8604: Posible argumento de referencia nulo para el parámetro "s" en "byte[] Encoding.GetBytes(string s)".
ReservasApi.Tests reutilizado correctamente (1,4s) + ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d45a1a (64-bit .NET 9.0.11)
[xUnit.net 00:00:00.06] Discovering: ReservasApi.Tests
[xUnit.net 00:00:00.09] Discovered: ReservasApi.Tests
[xUnit.net 00:00:00.09] Starting: ReservasApi.Tests
[xUnit.net 00:00:00.65] Finished: ReservasApi.Tests
ReservasApi.Tests prueba:realizado correctamente (2,5s)

Resumen de pruebas: total: 1; con errores: 0; correcto: 1; omitido: 0; duración: 2,4 s
Compilación correcto con 3 advertencias en 33,5s

C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet test --filter FullyQualifiedName=UsuariosControllerTests
Restauración completada (0,6s)
ReservasApi reutilizado correctamente (0,5s) + ReservasApi\bin\Debug\net9.0\ReservasApi.dll
ReservasApi.Tests reutilizado correctamente (2,4s) + ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d45a1a (64-bit .NET 9.0.11)
[xUnit.net 00:00:00.07] Discovering: ReservasApi.Tests
[xUnit.net 00:00:00.10] Discovered: ReservasApi.Tests
[xUnit.net 00:00:00.10] Starting: ReservasApi.Tests
[xUnit.net 00:00:00.62] Finished: ReservasApi.Tests
ReservasApi.Tests prueba:realizado correctamente (1,3s)

Resumen de pruebas: total: 1; con errores: 0; correcto: 1; omitido: 0; duración: 1,3 s
Compilación reutilizado correctamente en 0,2s

C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend>

```

Muestro ahora solo el último test: NEXT PAGE

```

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet test --filter FullyQualifiedName~UsuariosControllerTests
Restauración completada (0,6s)
ReservasApi realizado correctamente (0,5s) → ReservasApi\bin\Debug\net9.0\ReservasApi.dll
ReservasApi.Tests realizado correctamente (2,4s) → ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.11)
[xUnit.net 00:00:00.07] Discovering: ReservasApi.Tests
[xUnit.net 00:00:00.10] Discovered: ReservasApi.Tests
[xUnit.net 00:00:00.10] Starting: ReservasApi.Tests
[xUnit.net 00:00:00.62] Finished: ReservasApi.Tests
ReservasApi.Tests prueba realizado correctamente (1,3s)

Resumen de pruebas: total: 1; con errores: 0; correcto: 1; omitido: 0; duración: 1,3 s
Compilación realizado correctamente en 5,2s

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>

```

That output confirms:

1. Your test passed successfully.
2. Adding the `Assert.Contains` checks caused **no warnings**, because now the test actually verifies that the users returned include the specific `dni` values you seeded.
3. Your endpoint is correctly returning the expected users from the in-memory database.

So your `GET /api/usuarios` test is now **both passing and meaningful**: it checks the count and the specific entries.

You can follow the same pattern for other controller tests if you want to verify **specific properties** instead of just the count.

Ahora lo que voy a hacer es **añadir los `Assert.Contains` al test unitario `get api reservas` y hacer que use la clase `TestHelpers`** en vez de poner el código que esta contiene.

Mi nuevo código en **`ReservasControllerTests.cs`** es este:

```

1  using Microsoft.AspNetCore.Mvc;
2  using Microsoft.EntityFrameworkCore;
3  using ReservasApi.Controllers;
4  using ReservasApi.Data;
5  using ReservasApi.Modelos;
6  using ReservasApi.Modelos.DTO;
7
8  // using System.Linq;
9  // using System.Text;
10
11 namespace ReservasApi.Tests
12 {
13     0 references
14     public class ReservasControllerTests
15     {
16         /*
17          // Helper method to create a new in-memory DbContext
18          // Set up In-Memory AppDbContext for tests
19          // Each test should use a fresh in-memory database to avoid state sharing between tests.
20          private AppDbContext GetInMemoryDbContext() // le quité string dbName como parámetro
21          {
22              var opciones = new DbContextOptionsBuilder<AppDbContext>()
23                  .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString()) // dbName can be unique per test ("TestDb1", "TestDb2") to isolate them.
24                  .Options;
25              return new AppDbContext(opciones);
26          } // todo este metodo privado aparece greyed out while not yet used anywhere
27         */

```



```

28 // Test GET all reservas
29 [Fact] // each test method must have the [Fact] attribute for xUnit to recognize it.
0 references
30 public async Task GetReservas_RetornaAllReservas() // the test method must be inside the class that is marked public class ReservasControllerTests.
31 {
32     // Arrange: use TestHelpers to get an in-memory DbContext
33     using var contexto = TestHelpers.GetInMemoryDbContext(); // "TestDb1" // este método retorna un 'AppDbContext', que es el tipo de dato del parám
34
35     // Seed Usuarios:
36     // creo the related Usuario e Instalacion objects in the in-memory database before adding the Reserva:
37     var usuario1 = new Usuario { Dni = "12349876F", NombreCompleto = "Martin Gomez", Email = "mgomez@gmail.com", Telefono = "988765432", Domicilio = "
38     var usuario2 = new Usuario { Dni = "67895432C", NombreCompleto = "Jane Smith", Email = "jsmith@gmail.com", Telefono = "981735230", Domicilio = "
39     contexto.Usuarios.AddRange(usuario1, usuario2);
40
41     // Seed Instalaciones:
42     var instalacion1 = new Instalacion { InstalacionId = 1, Localizacion = "Pabellon A", CapacidadMaxima = 50, PrecioHora = 20.0, Tipo = true };
43     var instalacion2 = new Instalacion { InstalacionId = 2, Localizacion = "Pabellon B", CapacidadMaxima = 30, PrecioHora = 15.0, Tipo = false};
44     contexto.Instalaciones.AddRange(instalacion1, instalacion2);
45
46     // Seed Reservas:
47     contexto.Reservas.AddRange(
48         new Reserva
49         {
50             ReservaId = 1,
51             Dni = usuario1.Dni,
52             InstalacionId = instalacion1.InstalacionId,
53             NumeroAsistentes = 2,
54             FechaHora = new DateTime(2025, 12, 15, 10, 0, 0)
55         },
56         new Reserva
57         {
58             ReservaId = 2,
59             Dni = usuario2.Dni,
60             InstalacionId = instalacion2.InstalacionId,
61             NumeroAsistentes = 3,
62             FechaHora = new DateTime(2025, 12, 18, 15, 30, 0)
63         }
64     );
65
66     // contexto.Reservas.Add(new Reserva { ReservaId = 1, Dni = "12349876F", InstalacionId = 1, NumeroAsistentes = 2, FechaHora = new DateTime(2025, 12, 15, 1
67     // contexto.Reservas.Add(new Reserva { ReservaId = 2, Dni = "67895432C", InstalacionId = 2, NumeroAsistentes = 3, FechaHora = new DateTime(2025, 12, 18, 1
68
69     await contexto.SaveChangesAsync();
70
71     // instancio ReservasController pasándole el 'contexto' que justo antes inicialicé con el retorno de GetInMemoryDbContext.
72     var controlador = new ReservasController(contexto); // 'contexto0 es de tipo AppDbContext, que es el tipo del param del constructor de ReservasController
73
74     // Act:
75     var resultado = await controlador.GetReservas(); // GetReservas lo tengo definido en ReservasController.cs
76
77     // Assert adaptado al mi código de GetReservas() :
78     var okResultado = Assert.IsType<OkObjectResult>(resultado.Result); // idem assert genérico
79     var reservas = Assert.IsAssignableFrom<List<ReservaLeerDto>>(okResultado.Value); // cambio Reserva (assert genérico) por ReservaLeerDto
80
81     // Assets:
82     // Count:
83     Assert.Equal(2, reservas.Count); // idem assert genérico.
84
85     // Assert.Contains to verify each seeded Reserva exists:
86     Assert.Contains(reservas, r => r.ReservaId == 1 && r.Dni == usuario1.Dni && r.InstalacionId == instalacion1.InstalacionId);
87     Assert.Contains(reservas, r => r.ReservaId == 2 && r.Dni == usuario2.Dni && r.InstalacionId == instalacion2.InstalacionId);
88
89     } // final de [FACT] GetReservas_RetornaAllReservas()
90
91     // You can add more [Fact] methods here for POST, GET by ID, PUT, DELETE
92 } // final de la clase
93 } // final del namespace

```

Y el test ejecuta satisfactoriamente.

NEXT PAGE

```

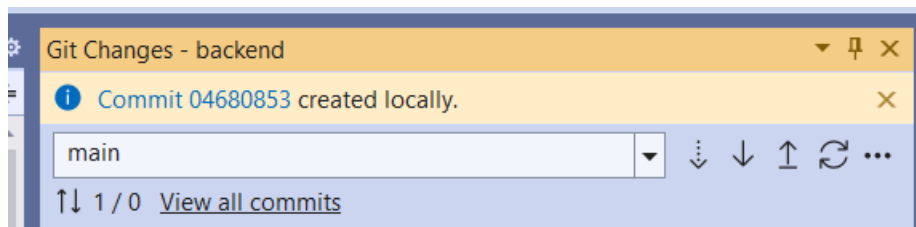
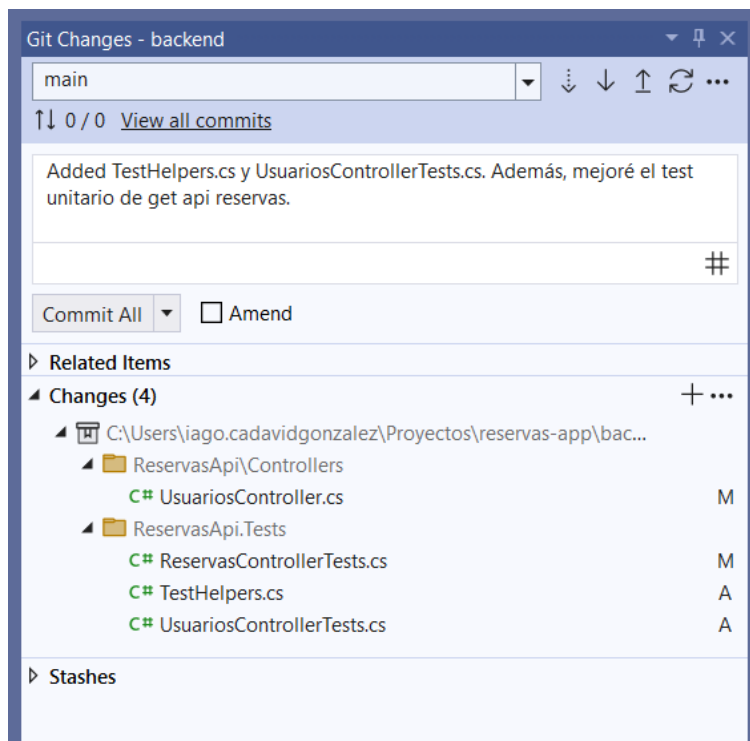
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend>dotnet test --filter FullyQualifiedName~ReservasControllerTests
Restauración completada (0,8s)
ReservasApi realizado correctamente (0,4s) → ReservasApi\bin\Debug\net9.0\ReservasApi.dll
ReservasApi.Tests realizado correctamente (2,7s) → ReservasApi.Tests\bin\Debug\net9.0\ReservasApi.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v2.8.2+699d445a1a (64-bit .NET 9.0.11)
[xUnit.net 00:00:00.07] Discovering: ReservasApi.Tests
[xUnit.net 00:00:00.09] Discovered: ReservasApi.Tests
[xUnit.net 00:00:00.10] Starting: ReservasApi.Tests
[xUnit.net 00:00:00.71] Finished: ReservasApi.Tests
ReservasApi.Tests prueba realizado correctamente (1,5s)

Resumen de pruebas: total: 1; con errores: 0; correcto: 1; omitido: 0; duración: 1,5 s
Compilación realizado correctamente en 5,9s

```

Entonces a estas alturas ya tenemos tests xunit exitosos para get api reservas y get api usuarios, siendo el primero más complejo de codificar que el segundo porque el primero tiene dos foreign key requeridas y además en su controller lo que se retorna es DTO en vez de directamente la entidad.

Como no hay tales fks en el caso de tabla usuarios y en su controller en relación al get api usuarios se retorna directamente la entidad, pues entonces el código de xUnit test get api usuarios es más simple.



NEXT PAGE

